Data Visualization in R

Workshop Lead: Meghana Munipalle

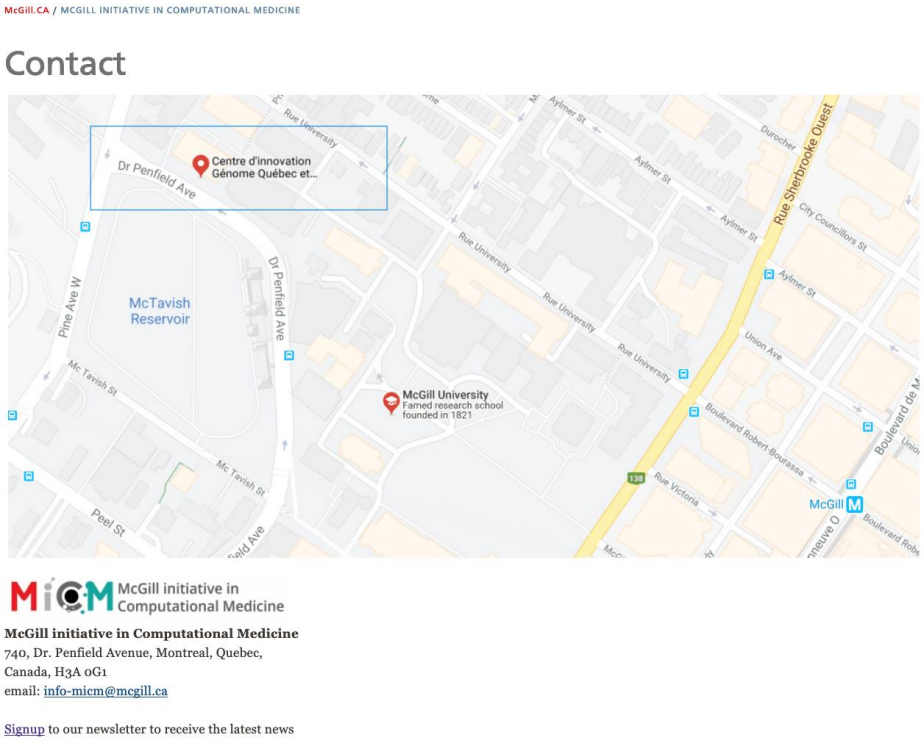
Date: October 27, 2026

McGill initiative in Computational Medicine

Mission : aims to deliver inter-disciplinary research programs and empower the use of data in health research and health care delivery

Scan the QR code to sign up for our **mailing list!**

https://www.mcgill.ca/micm



Goals

- **Better interpretability:** transformations & faceting to make better comparisons
- **Better clarity:** customize your aesthetics to communicate data insights more effectively
- **Better engagement:** let people interact with your visualizations
- **Better storytelling:** what's the right plot type for your analysis?

Outline



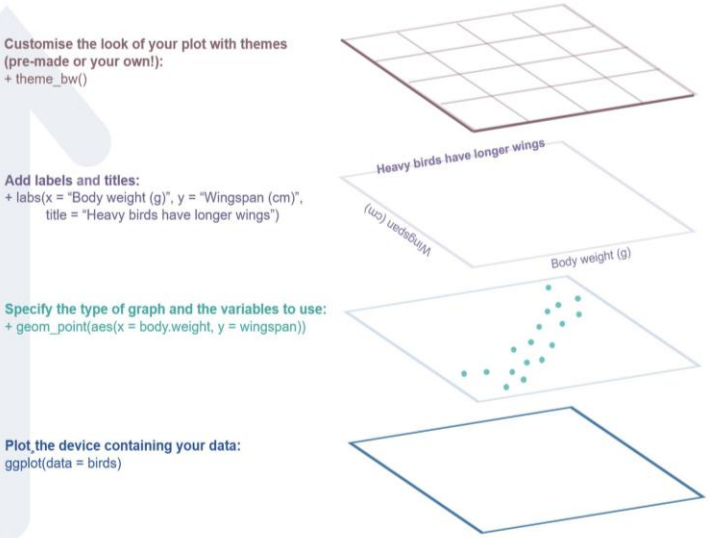
- **Module 1: Brief Review of Basic Visualization in R**
  - Why use ggplot 2
  - Structure of a ggplot2 command
- **Module 2: Advanced Customization and Data Transformation**
  - Data transformations and faceting
  - Customizing ggplot2 plots
  - *Exercise 1*
- **Module 3: Advanced Visualization Techniques**
  - Specialized plots
  - Introduction to interactive and dynamic visualizations
  - *Exercise 2*

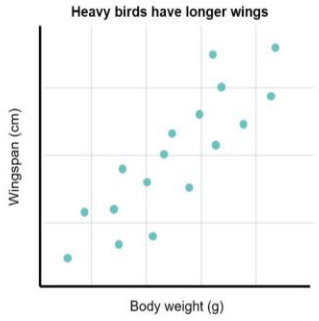
Why Use ggplot2 Over Base R for Data Visualization?



Feature	ggplot2	Base R
Design Framework	Based on Grammar of Graphics, enabling **layered** construction	Procedural approach, **harder** to modify and extend
Customization	Themes and formatting (theme_*) for professional aesthetics	Requires **manual** adjustments using par() or axis()
Faceting	**Built-in** with facet_wrap() and facet_grid()	Requires **manual** layout setup
Syntax	Consistent and modular, **easy** to learn and maintain	**Fragmented** and less intuitive
Data Transformations	**Directly** supports scaling and slicing in plots	**Preprocessing required** before visualization
Multi-layered Plots	**Allows layering** of multiple plot types and elements	**Complex** to achieve with overlays
Interactivity	**Seamless integration** with interactive packages like plotly	Requires **external packages** with complex workflows

Overview of ggplot2

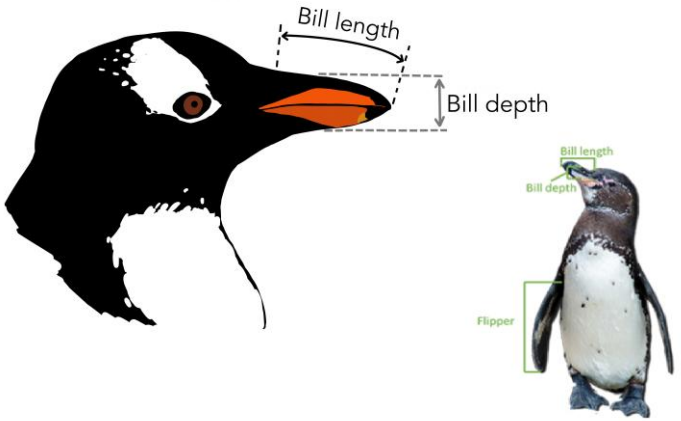




- gg = “grammar of graphics”
- Components in ggplot2:
  - **Data**
  - **Aesthetic attribute (Aes)**
  - **Geometric object (Geom)**
  - Stats
  - Position
  - Coord
  - Facet
  - Theme
  - ……

```
ggplot(data = <DATA>) +
  <GEOM_FUNCTION>(
     mapping = aes(<MAPPINGS>),
     stat = <STAT>,
     position = <POSITION>
  ) +
  <COORDINATE_FUNCTION> +
  <FACET_FUNCTION>
```

Structure of the ggplot2 command



What is the relationship between penguin bill length and bill depth?

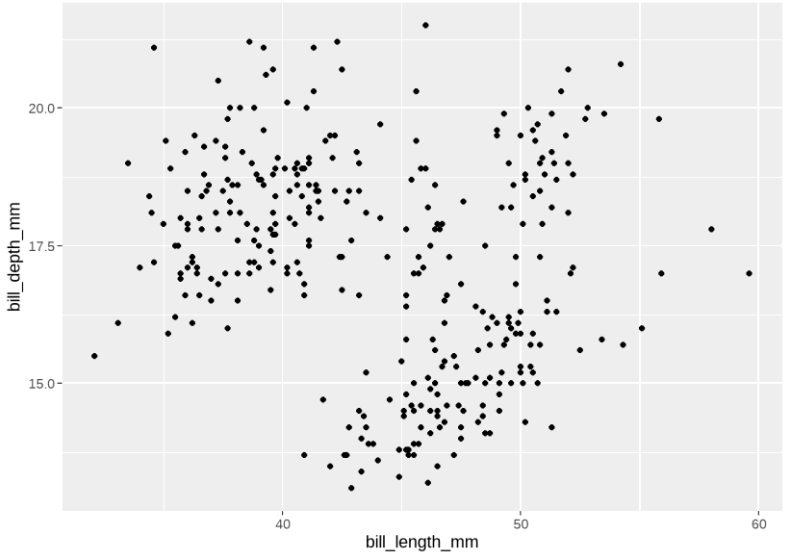
data set

```
ggplot( data = penguins ) +
```

aes() x variable y variable

```
geom_point( aes(x = bill_length_mm,  y = bill_depth_mm) )
```

"type" of layer



Dataset preparation

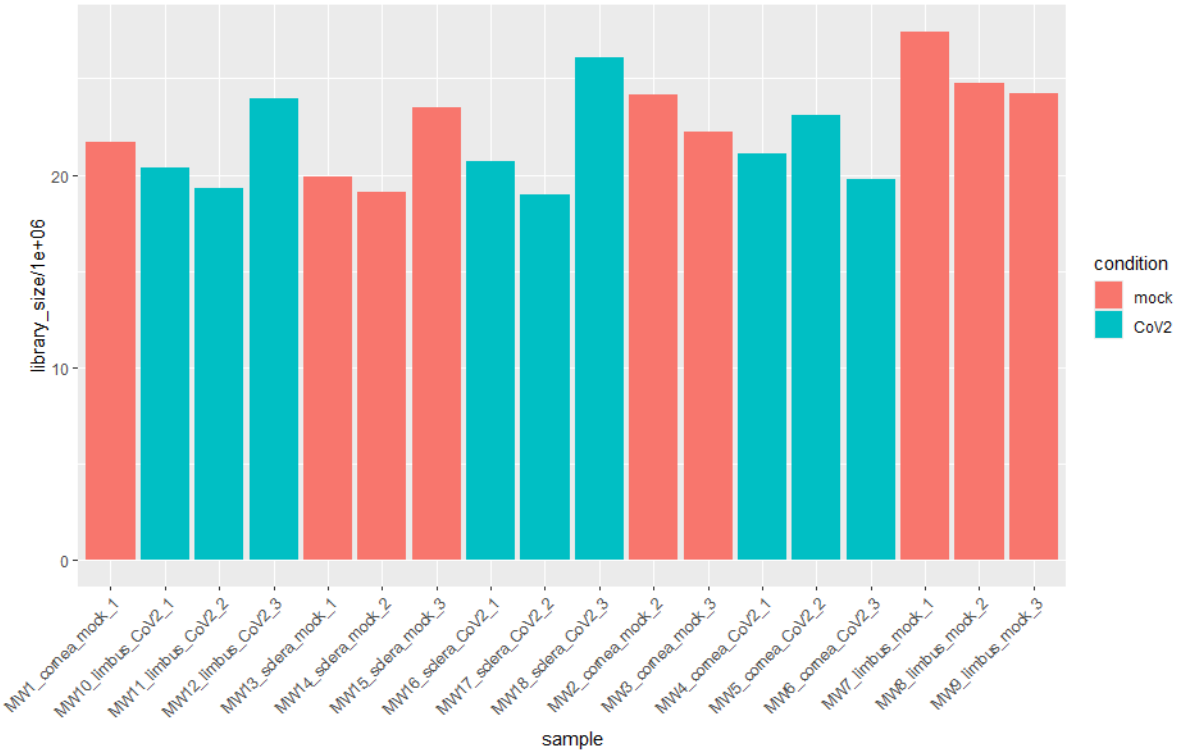
RNA-seq

- 387,216 observations
- 9 variables
  - **Gene**: gene symbol (about 21,500 expressed genes)
  - **sample**: full sample name, a factor with 18 level
  - **id**: sample ID from the original study (MW1–MW18)
  - **tissue**: a factor with 3 levels (cornea, limbus, sclera)
  - **condition**: a factor with 2 levels (mock, CoV2)
  - **replicate**: 1–3 per tissue × condition group
  - **count**: raw number of sequencing reads mapped to the gene
  - **cpm**: counts per million, which adjusts for each sample's sequencing depth
  - **log_cpm**: log2(cpm + 1), compressing the very skewed range of expression values

	Gene	id	tissue	condition	replicate	sample	count	cpm	log_cpm
1	A1BG	MW1	cornea	mock	1	MW1_cornea_mo...	91	4.19914467	2.37827430
2	A1BG	MW2	cornea	mock	2	MW2_cornea_mo...	131	5.42230101	2.68309029
3	A1BG	MW3	cornea	mock	3	MW3_cornea_mo...	86	3.86513449	2.28247969
4	A1BG	MW4	cornea	CoV2	1	MW4_cornea_Co...	77	3.64370025	2.21527485
5	A1BG	MW5	cornea	CoV2	2	MW5_cornea_Co...	69	2.98590032	1.99490563

Bar plot

- discrete variable
- ggplot(samples, aes(x = sample, y = library_size / 1e6, fill = condition)) + geom_col() + theme(axis.text.x = element_text(angle = 45, hjust = 1), plot.margin = margin(t = 5, r = 5, b = 5, l = 40))

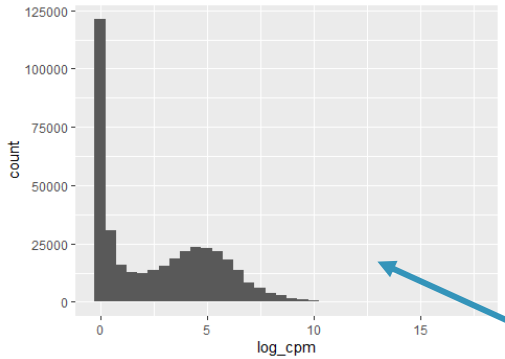


Research Question:
Were all 18 samples sequenced to a similar depth?

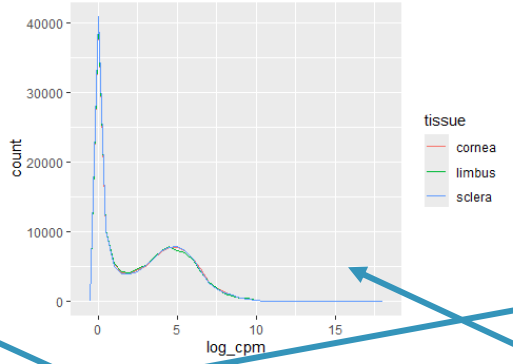
Histogram/ Frequency/ Density Plot

Choosing the right plot
- To see **raw counts** and **distribution shape** -> Histogram
- To compare **multiple groups** without overlap issues -> Frequency
- To observe **smoothed probability distributions** and **peaks** -> Density

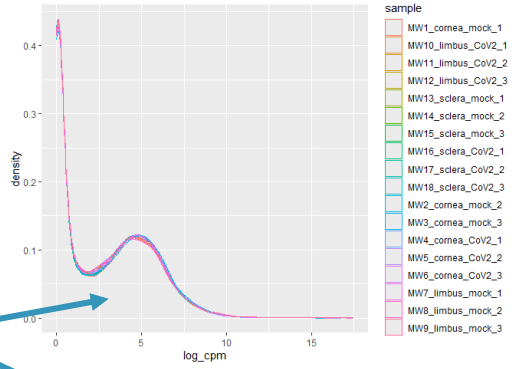
- ggplot(expr, aes(x = log_cpm)) + geom_histogram(binwidth = 0.5)



- ggplot(expr, aes(x = log_cpm, colour = tissue)) + geom_freqpoly(binwidth = 0.5)



- ggplot(expr, aes(x = log_cpm, colour = sample)) + geom_density()



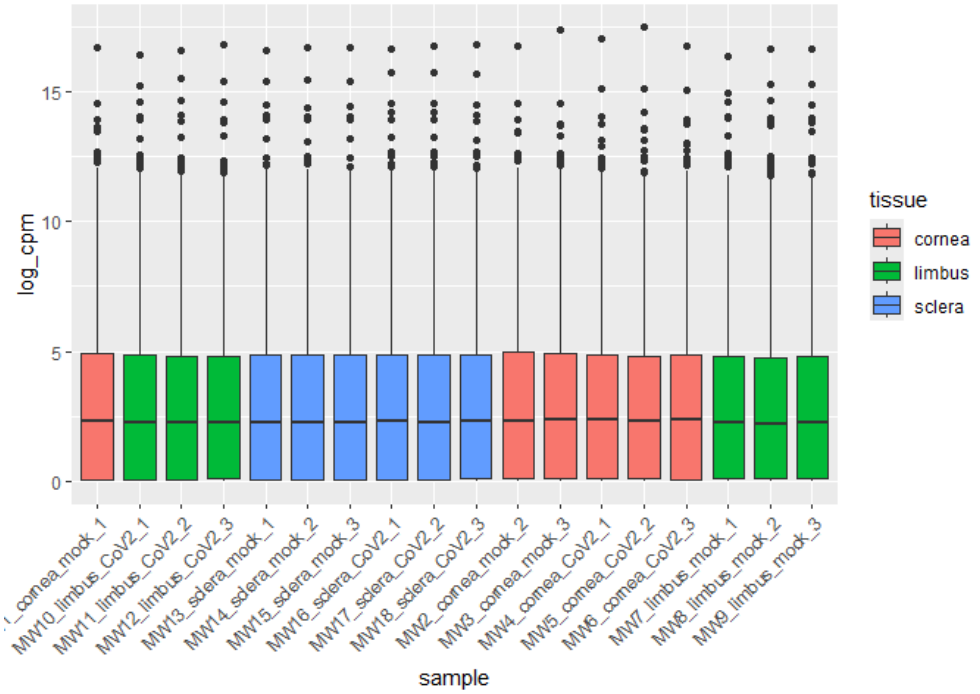
Do all samples have similar expression distributions, or does any sample look unusual?

What is the overall distribution of gene expression levels?

Compare the expression distributions across the three tissues.

Box plot

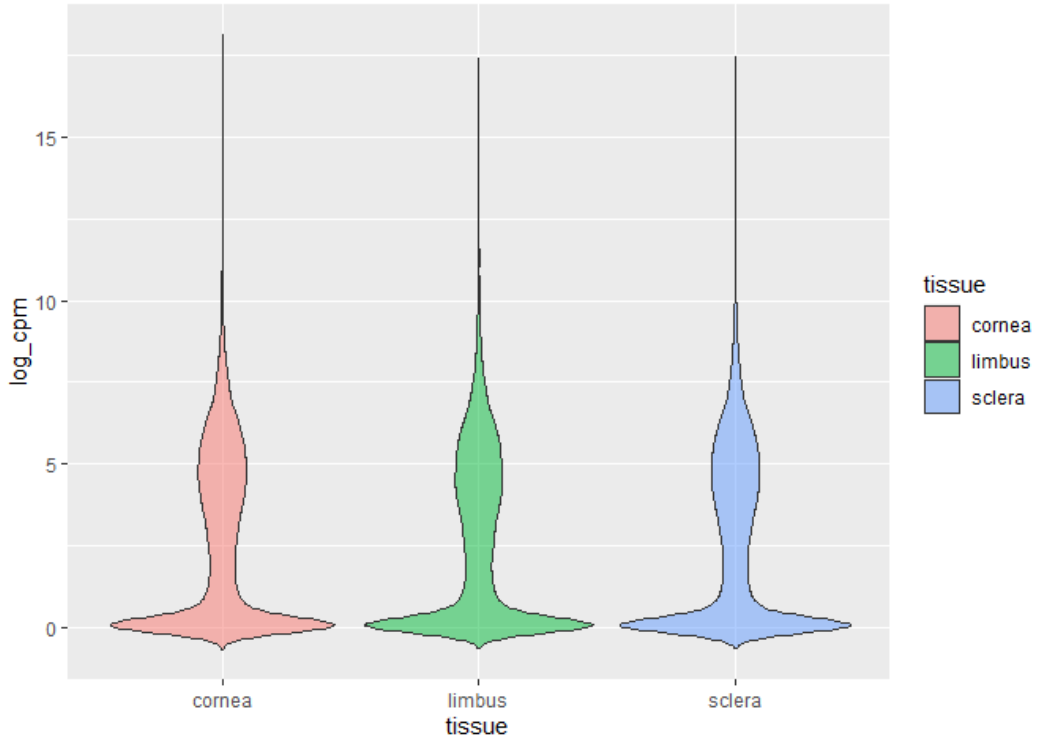
- one discrete variable + one continuous variable
- ggplot(expr, aes(x = sample, y = log_cpm, fill = tissue)) + geom_boxplot() + theme(axis.text.x = element_text(angle = 45, hjust = 1))



Research Question:
How does the distribution of gene expression differ between samples?

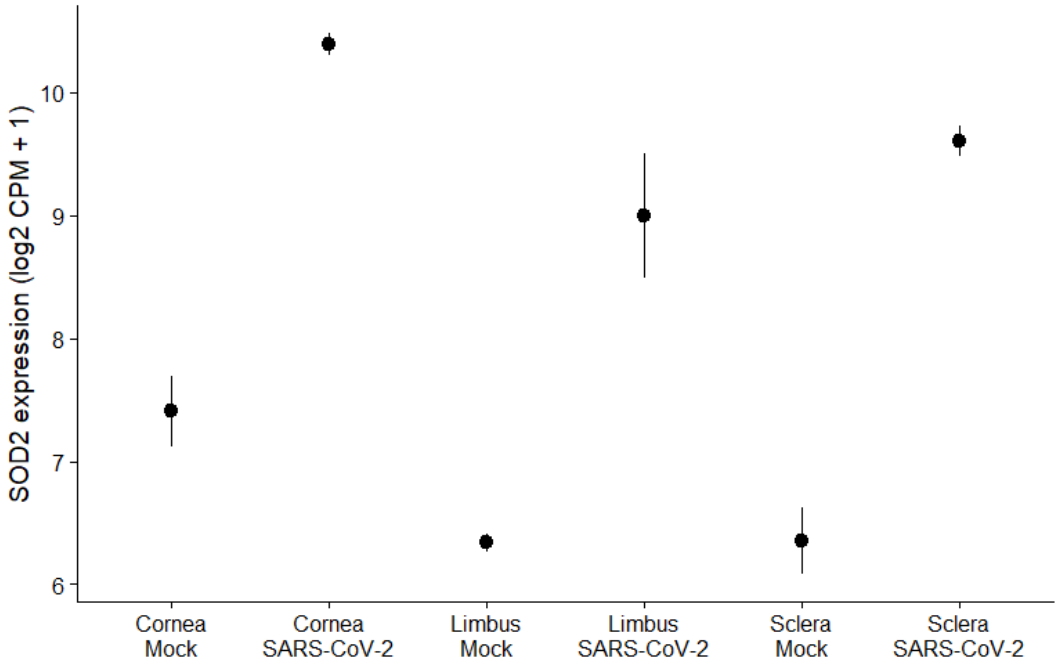
Violin plot

- A combination of a box plot and a density plot
- ggplot(expr, aes(x = tissue, y = log_cpm, fill = tissue)) + geom_violin(trim = FALSE, alpha = 0.5)

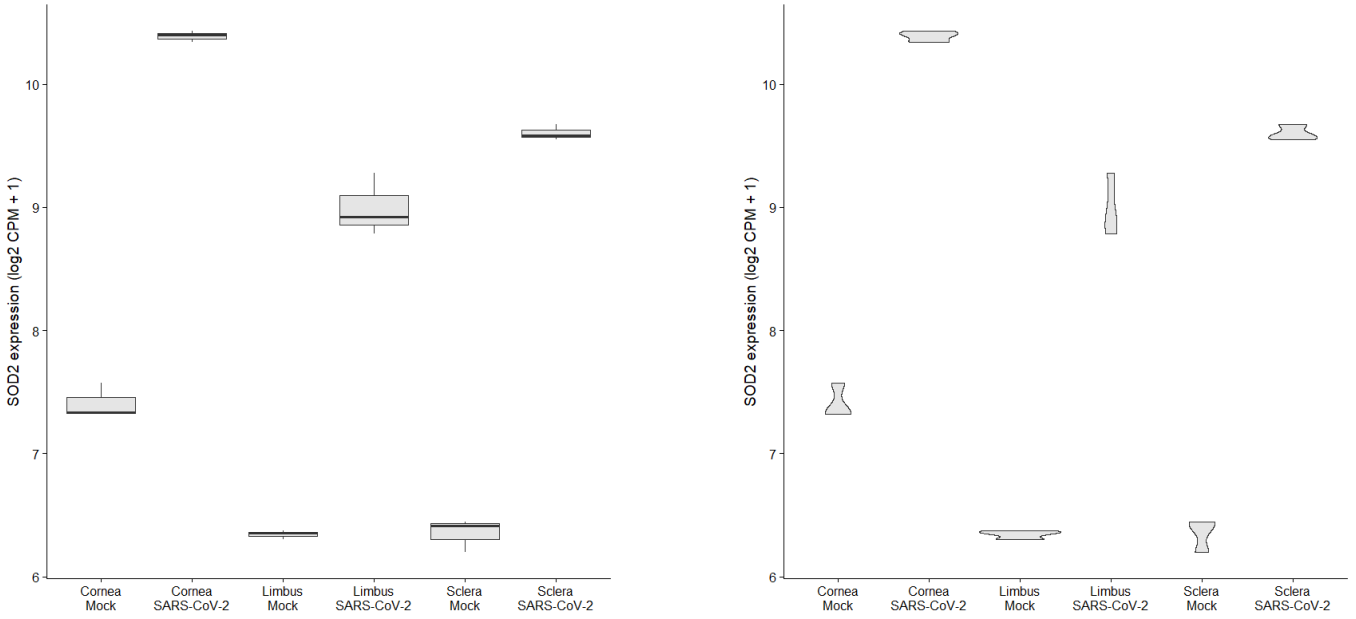


Research Question:
How does the distribution and variability of gene expression differ across tissues?

What questions do you have about this plot?

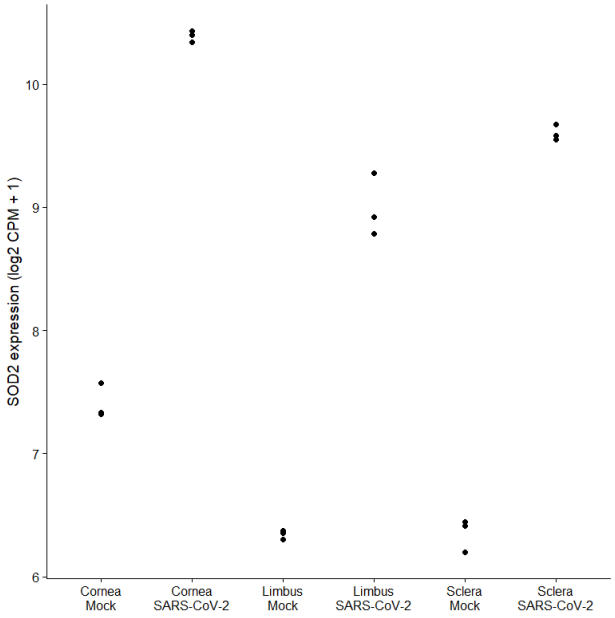


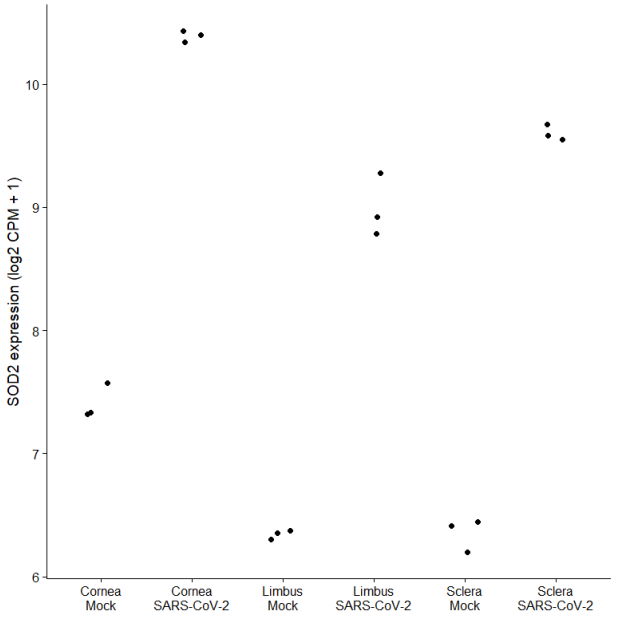
Next step: box/violin plot to show the distribution



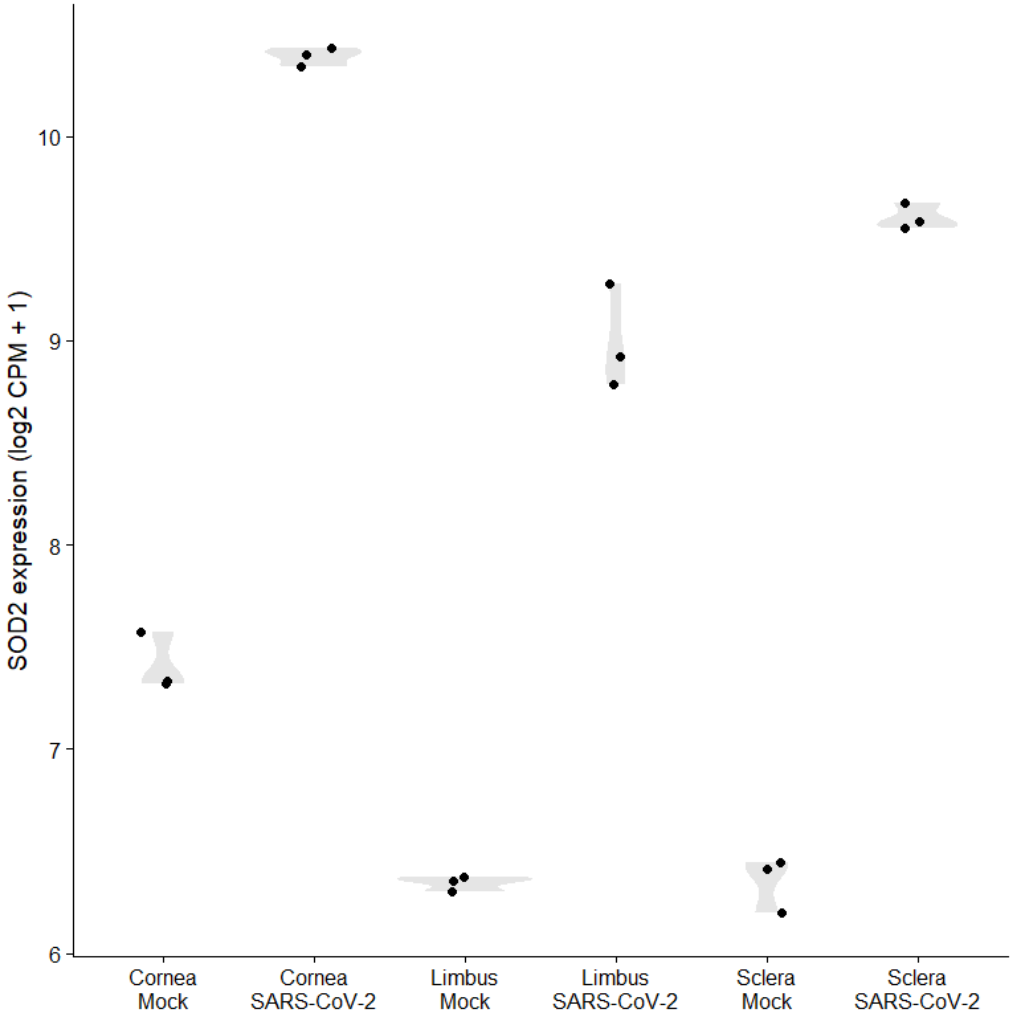
What is the problem of the violin or box plot?

Show raw data points

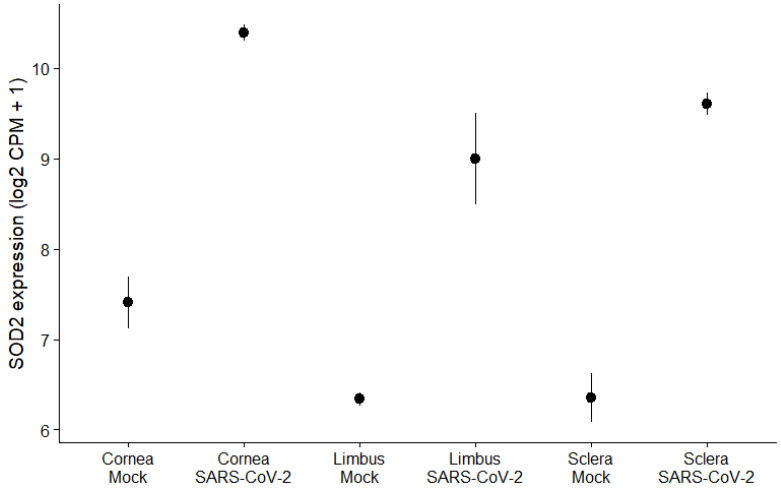


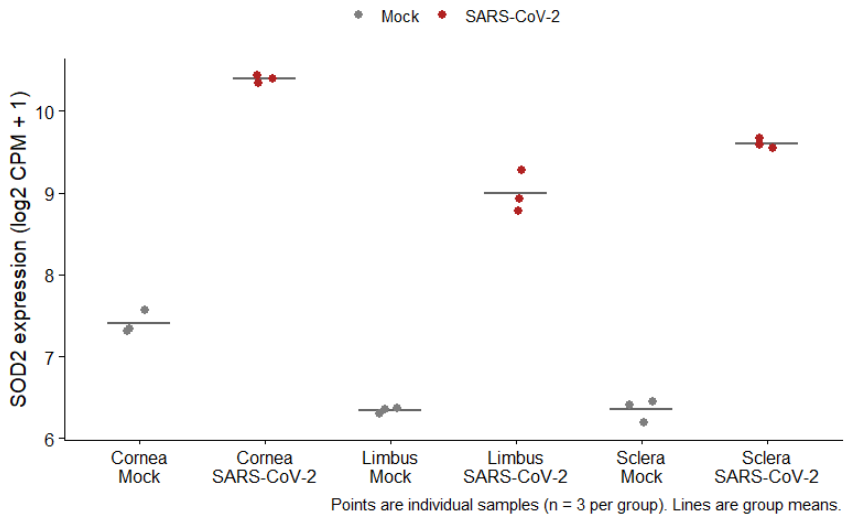


Combined…?


SOD2 expression (log2 CPM + 1)
10
9
8
7
6
Cornea Mock
Cornea SARS-CoV-2
Limbus Mock
Limbus SARS-CoV-2
Sclera Mock
Sclera SARS-CoV-2

Before and after







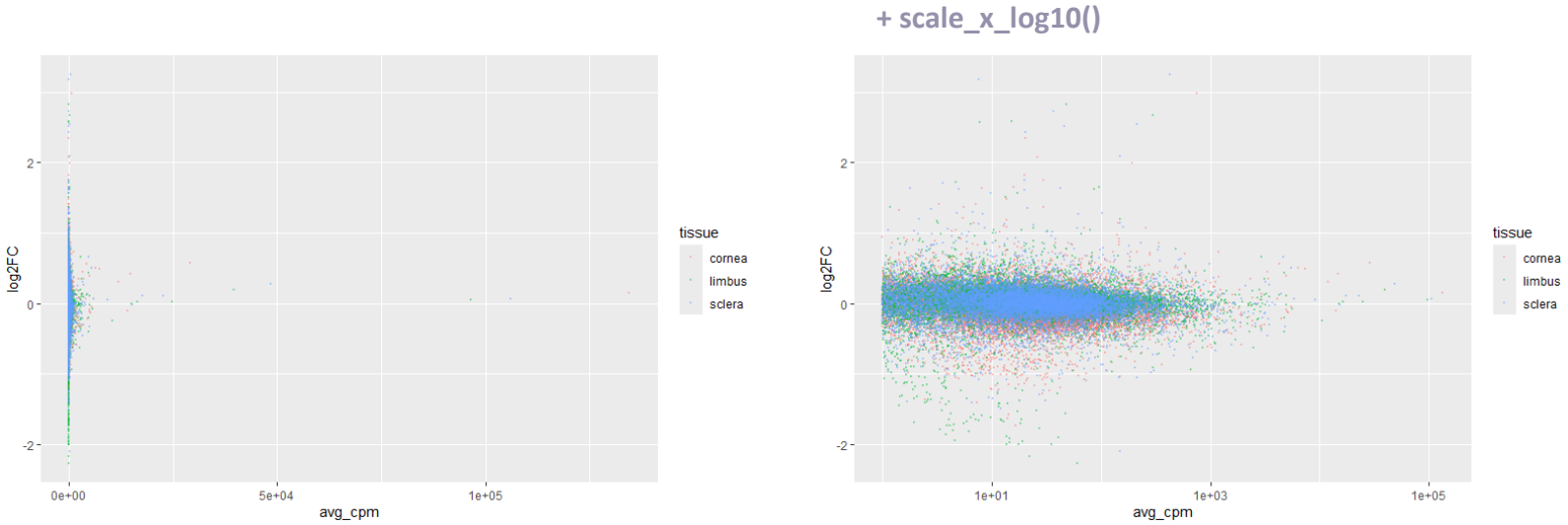
Data Transformation and Advanced Customization

Data Transformation

	Why?	Functions
Data reshaping	Useful for faceting or grouping variables	tidyr::pivot_longer() tidyr::pivot_wider()
Filtering rows and select columns	To focus on relevant data for a clearer visualization. To remove noise or unnecessary categories.	dplyr::filter() dplyr::select()
Aggregation and summarization	Transforming raw data into summarized statistics for better visualization.	dplyr::group_by() dplyr::summarize()
Normalization and scaling	Adjusting numerical values to ensure fair comparisons. Log transformation Min-max scaling Z-score standardization	dplyr::mutate() scale_x_log10()
Binning (Discretization)	Dividing a continuous variable into discrete categories to visualize distributions more clearly (e.g., in histograms or heatmaps).	dplyr::mutate()

Data Transformation – log transformation

- Research Question: How does a gene's average expression level relate to how much it changes after SARS-CoV-2 infection, across different tissues?

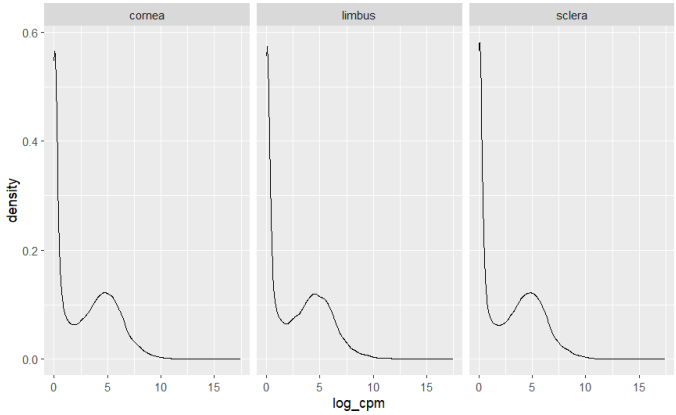


Data faceting

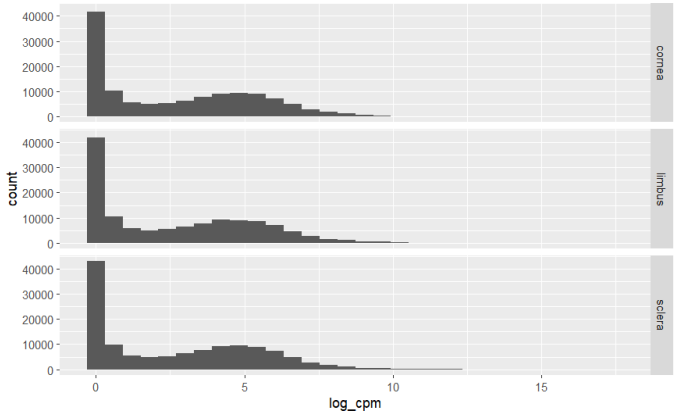
Facet_grid()

create a grid of graphs, by rows and columns

+ facet_grid(. ~ tissue)



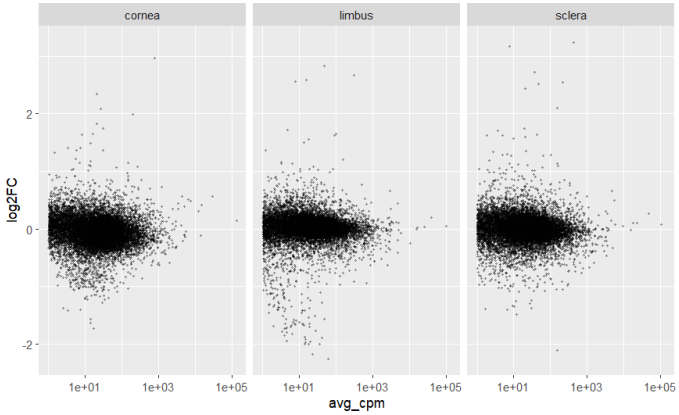
+ facet_grid(tissue ~ .)



Facet_wrap()

create small multiples by "wrapping" a series of plots

+ facet_wrap(~tissue)



Choose grid:

- Compare categorical variables systematically
- The number of facet levels is not too large

Choose wrap:

- Flexible and space-efficient

Aesthetic customizations – color palettes

colorspace



Qualitative: categorical variables
qualitative_hcl()

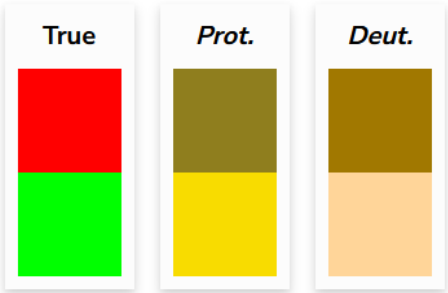
Sequential: ordered/continuous numerical variables
sequential_hcl()

Diverging: ordered/continuous numerical variables
diverging_hcl()

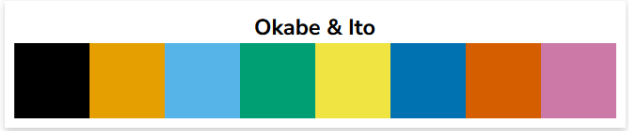
Don't use bidirectional (diverging) color scales for unidirectional data!

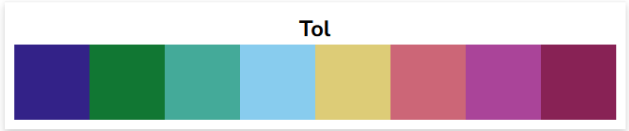
Colorblindness-friendly palettes

The three types of cone cell in your eye are each responsible for detecting a different primary color: red, green, and blue. Because anomalies in these cells interfere with a person's ability to see these colors normally, it is a bad idea to use a color palette of red, green, and blue. In particular, you should avoid the common practice of using red and green as contrasting colors. See why for yourself.









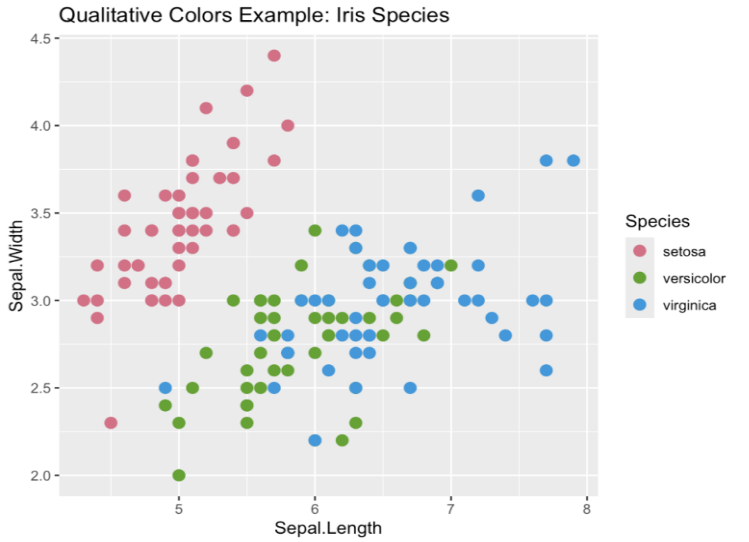
https://davidmathlogic.com/colorblind/#%23D81B60-%231E88E5-%23FFC107-%23004D40
https://cran.r-project.org/web/packages/khroma/vignettes/tol.html

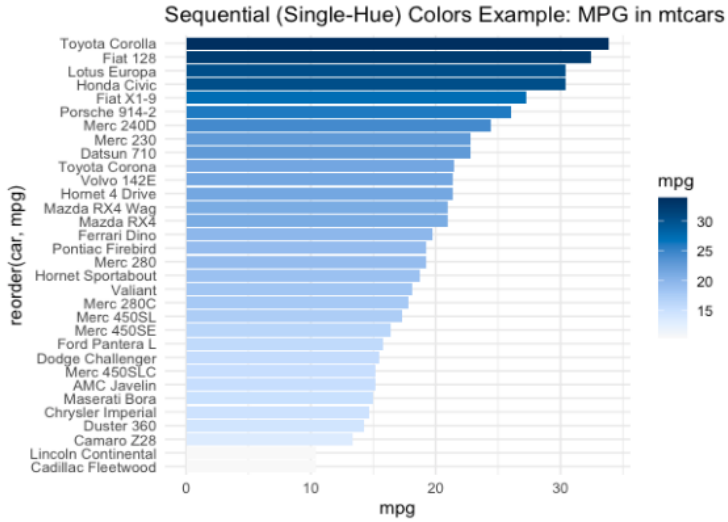
How to use colorspace in ggplot2

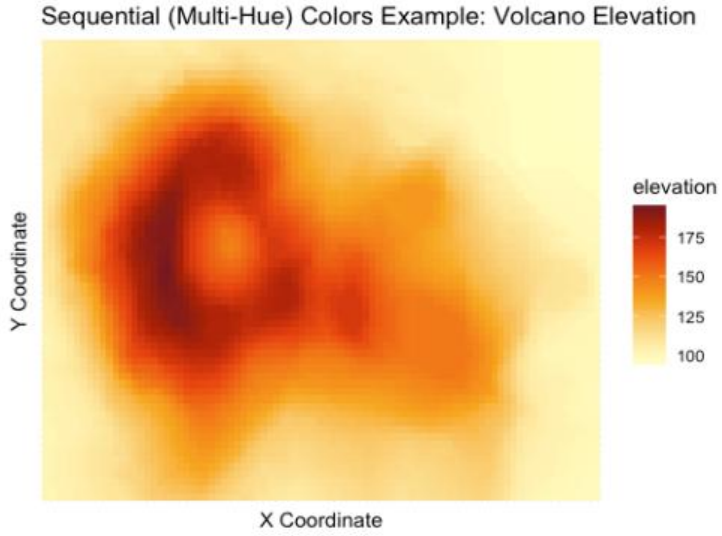
scale_<aesthetic>_<datatype>_<colorscale>()

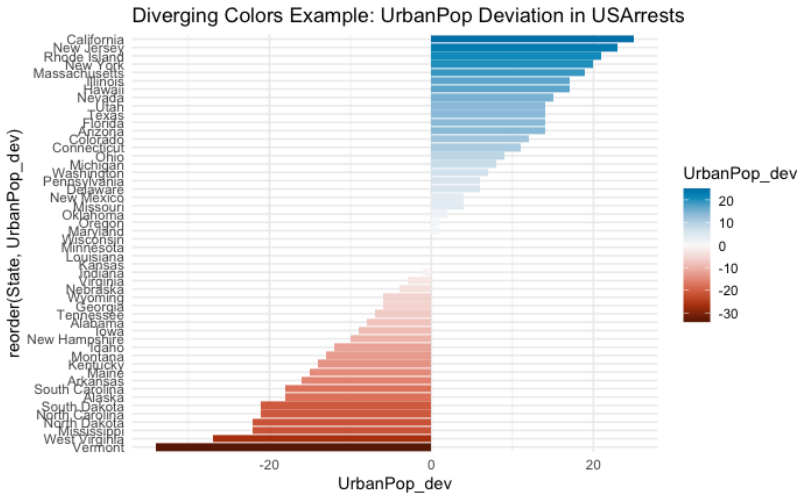
Scale function	Aesthetic	Data type	Palette type
`scale_color_discrete_qualitative()`	`color`	discrete	qualitative
`scale_fill_continuous_sequential()`	`fill`	continuous	sequential
`scale_colour_continuous_divergingx()`	`colour`	continuous	diverging

Colorspace examples





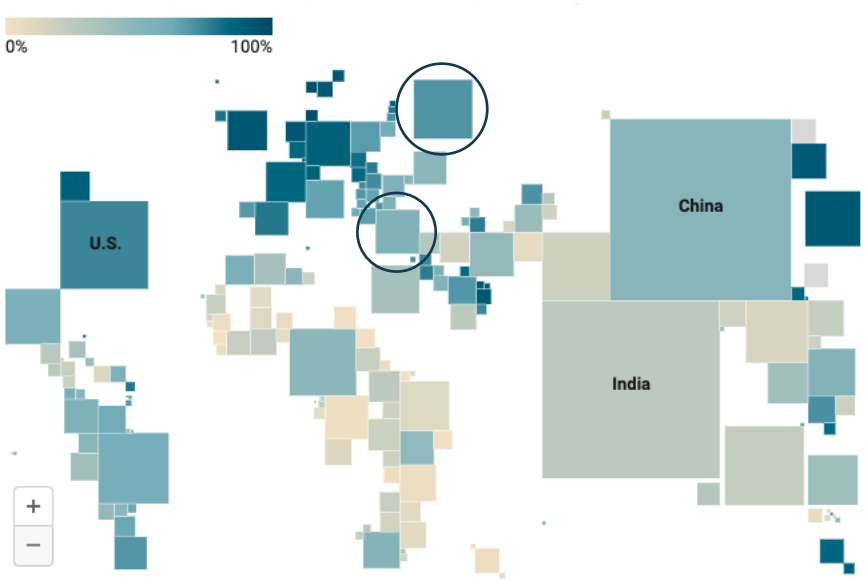


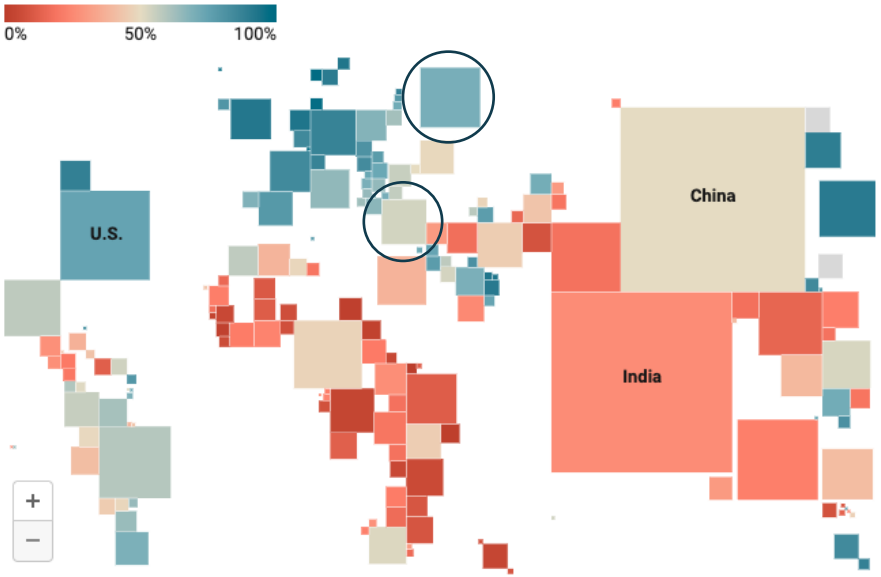


How to choose diverging and sequential color scales?

Use diverging scales IF…

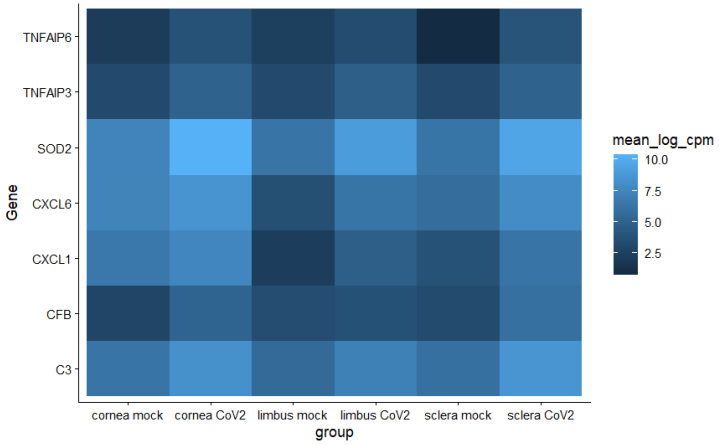
- …there's a **meaningful middle point**
- …you want to **emphasize the extremes**
- …you want to let readers **see more differences** in the data



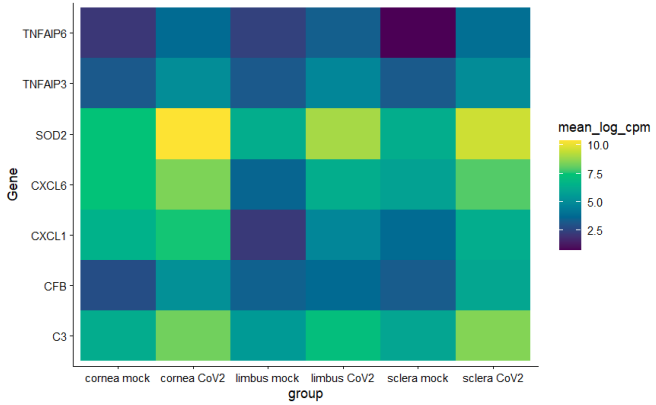


https://blog.datawrapper.de/diverging-vs-sequential-color-scales/

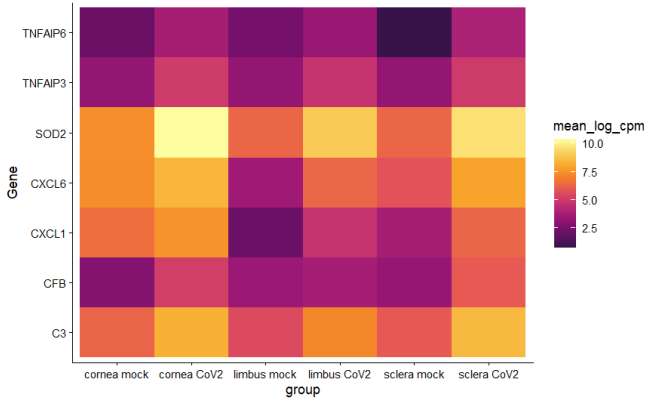
Select the appropriate color palettes for heatmaps



scale_fill_continuous_sequential(palette = "Viridis", rev = FALSE)



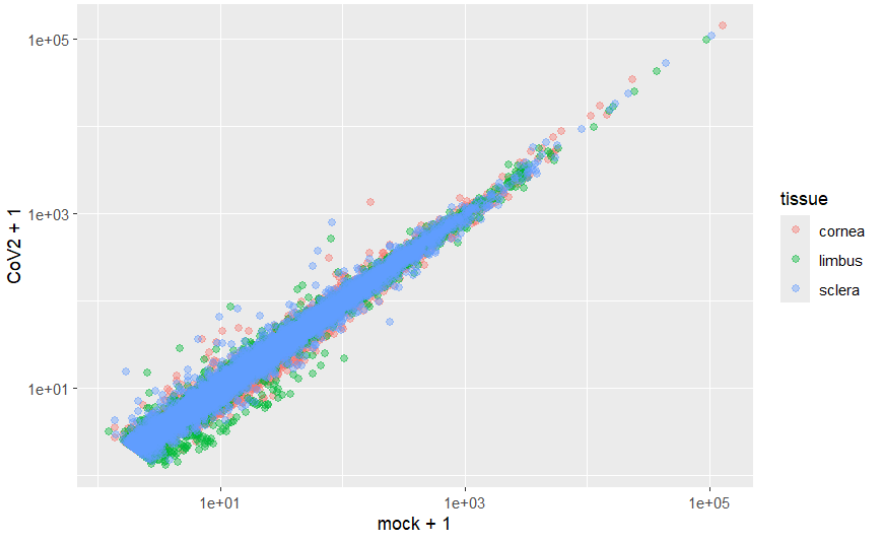
scale_fill_continuous_sequential(palette = ”Inferno", rev = FALSE)



How to change size of points and transparency

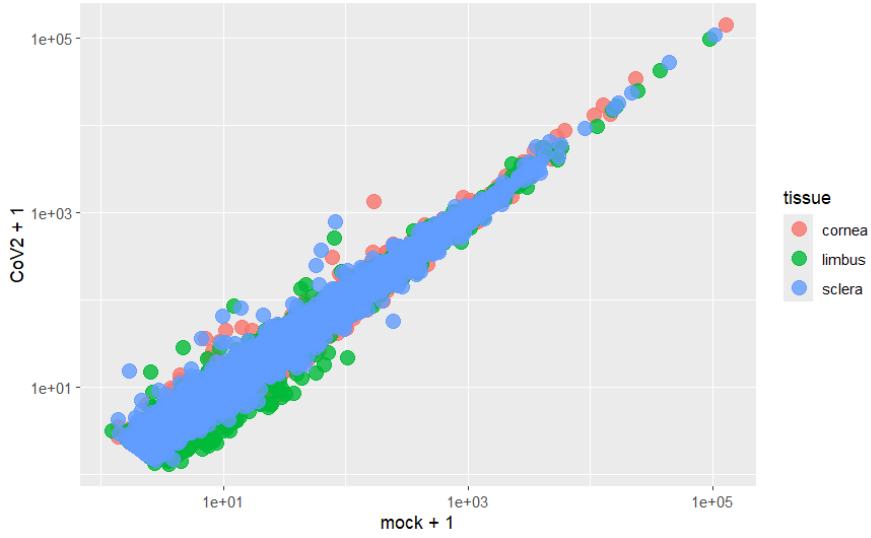
Size of points: size

```
ggplot(gene_summary) +
  geom_point(aes(x = mock + 1,
                 y = CoV2 + 1,
                 color = tissue),
             size = 2,
             alpha = 0.4) +
  scale_x_log10() +
  scale_y_log10()
```



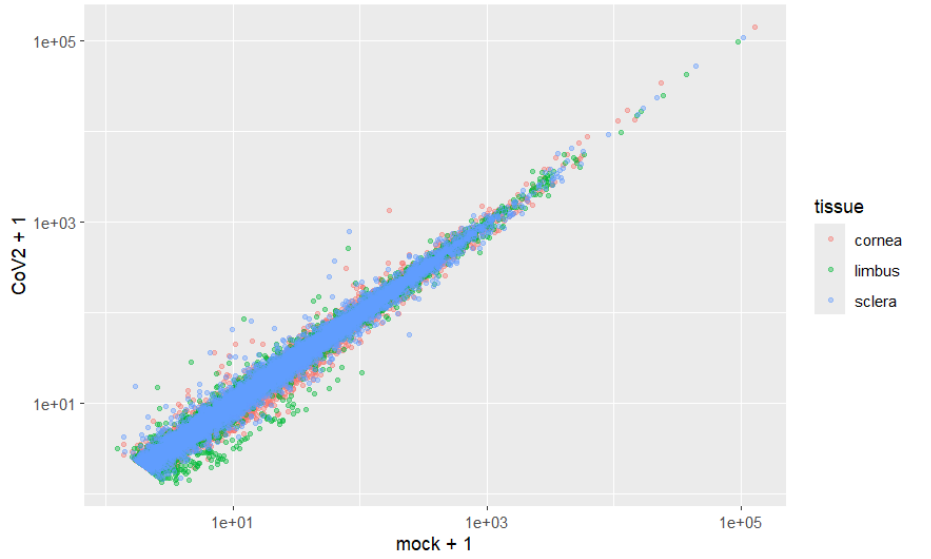
Transparency:
alpha = number between 0 (completely transparent) and 1 (completely opaque)

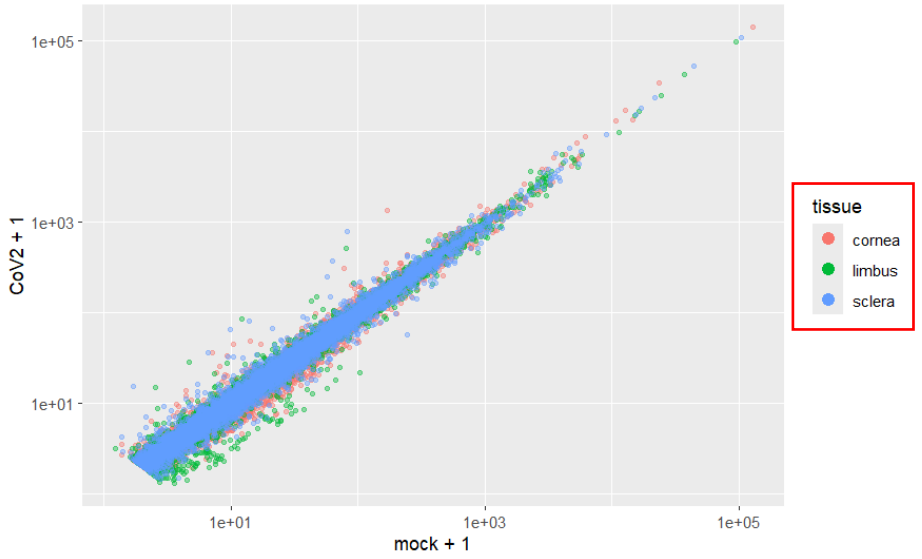
```
ggplot(gene_summary) +
  geom_point(aes(x = mock + 1,
                 y = CoV2 + 1,
                 color = tissue),
             size = 4,
             alpha = 0.8) +
  scale_x_log10() +
  scale_y_log10()
```



Adjust the appearance of the legend

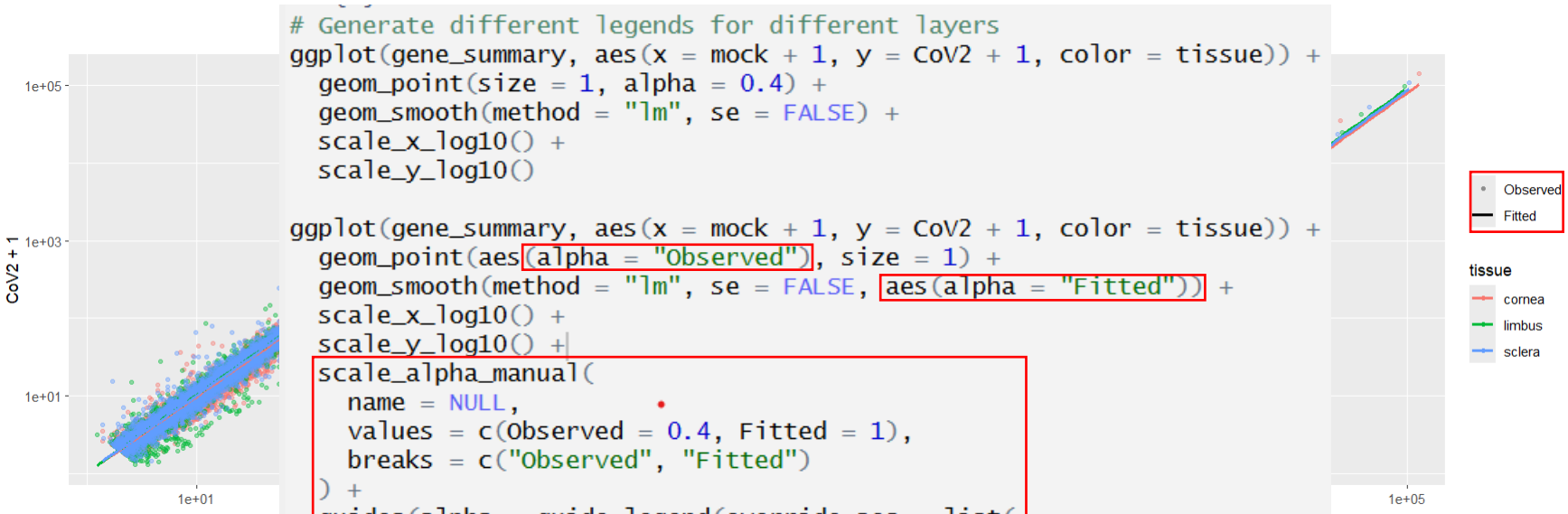
```
+ guides(color = guide_legend(
                override.aes = list(size = 3, alpha = 1))
        )
```





Generate different legends for different layers

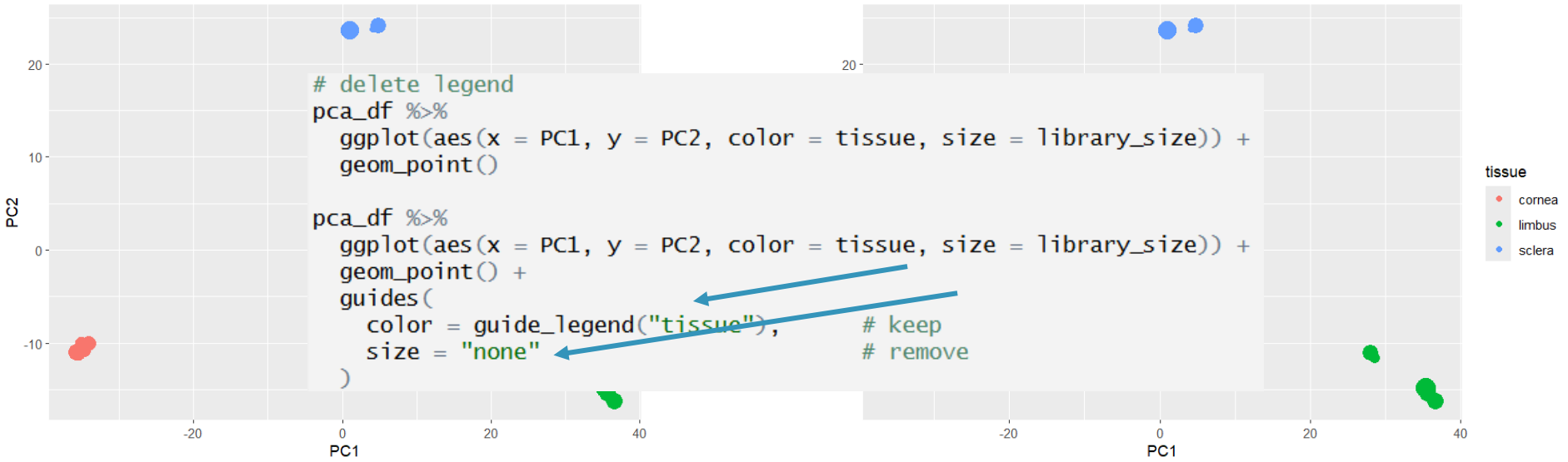
How to add a legend to illustrate the original observations and the fitted line?




```r
# Generate different legends for different layers
ggplot(gene_summary, aes(x = mock + 1, y = CoV2 + 1, color = tissue)) +
  geom_point(size = 1, alpha = 0.4) +
  geom_smooth(method = "lm", se = FALSE) +
  scale_x_log10() +
  scale_y_log10()

ggplot(gene_summary, aes(x = mock + 1, y = CoV2 + 1, color = tissue)) +
  geom_point(aes(alpha = "Observed"), size = 1) +
  geom_smooth(method = "lm", se = FALSE, aes(alpha = "Fitted")) +
  scale_x_log10() +
  scale_y_log10() +
  scale_alpha_manual(
    name = NULL,
    values = c(Observed = 0.4, Fitted = 1),
    breaks = c("Observed", "Fitted")
  ) +
  guides(alpha = guide_legend(override.aes = list(
    linetype = c(0, 1),
    shape = c(16, NA),
    color = "black"
  )))
```

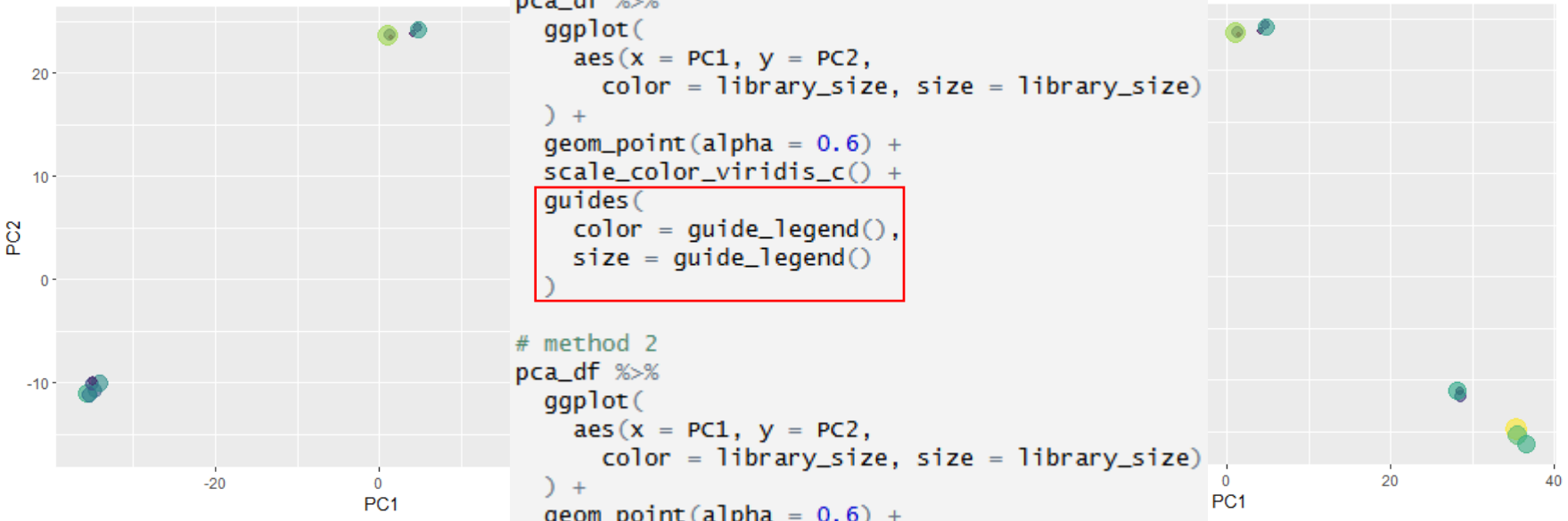
Delete the legend




```
# delete legend
pca_df %>%
  ggplot(aes(x = PC1, y = PC2, color = tissue, size = library_size)) +
  geom_point()

pca_df %>%
  ggplot(aes(x = PC1, y = PC2, color = tissue, size = library_size)) +
  geom_point() +
  guides(
    color = guide_legend("tissue"),      # keep
    size = "none"                        # remove
  )
```

Merge the legend



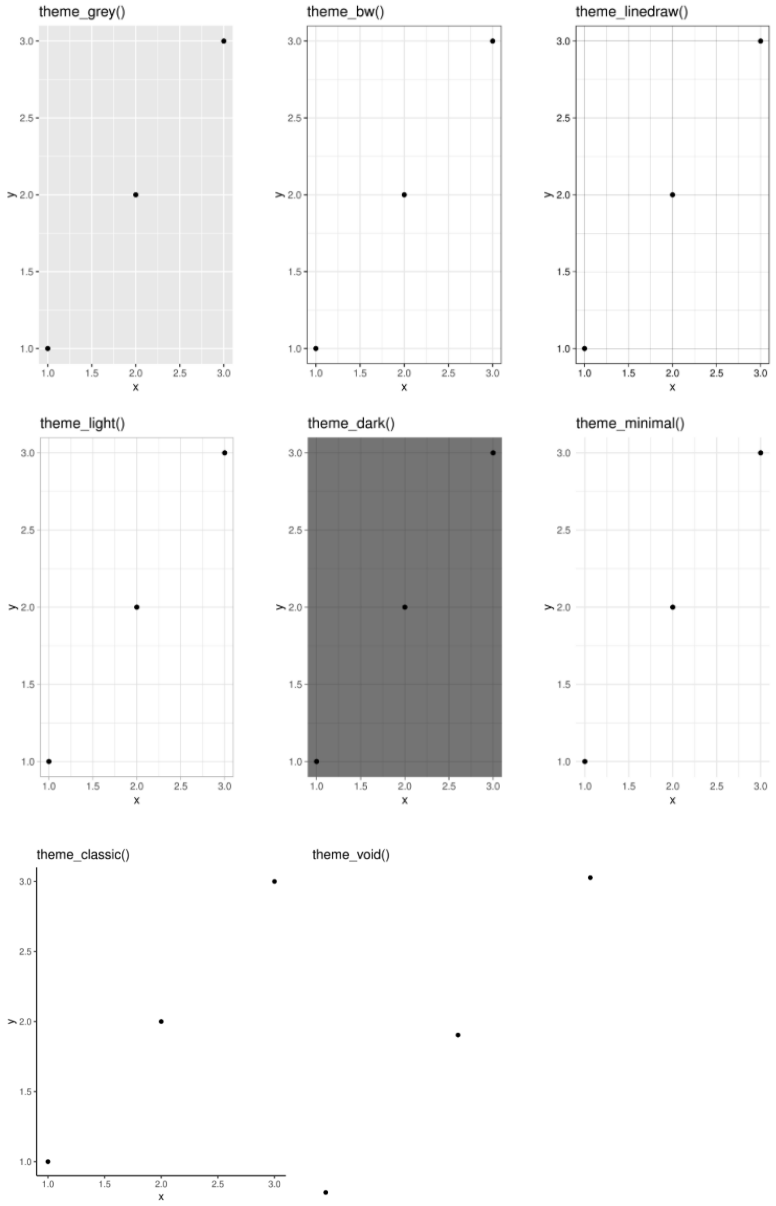

```r
# method 1
pca_df %>%
  ggplot(
    aes(x = PC1, y = PC2,
      color = library_size, size = library_size)
  ) +
  geom_point(alpha = 0.6) +
  scale_color_viridis_c() +
  guides(
    color = guide_legend(),
    size = guide_legend()
  )

# method 2
pca_df %>%
  ggplot(
    aes(x = PC1, y = PC2,
      color = library_size, size = library_size)
  ) +
  geom_point(alpha = 0.6) +
  scale_color_viridis_c(guide = "legend")
```

Theme

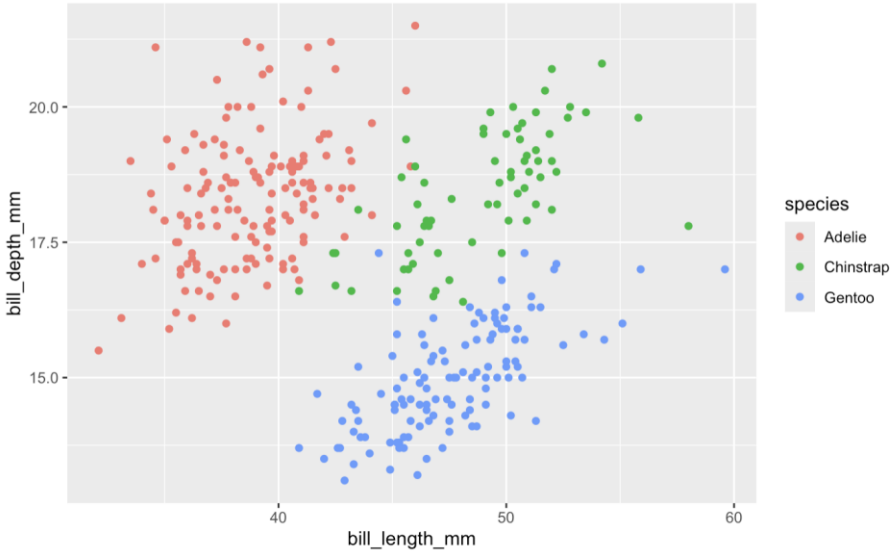
- df <- data.frame(x = 1:3, y = 1:3)
- base <- ggplot(df, aes(x, y)) + geom_point()

- base + **theme_grey()** + ggtitle("theme_grey()")
- base + **theme_bw()** + ggtitle("theme_bw()")
- base + **theme_linedraw()** + ggtitle("theme_linedraw()")
- base + **theme_light()** + ggtitle("theme_light()")
- base + **theme_dark()** + ggtitle("theme_dark()")
- base + **theme_minimal()** + ggtitle("theme_minimal()")
- base + **theme_classic()** + ggtitle("theme_classic()")
- base + **theme_void()** + ggtitle("theme_void()")



Exercise 1 (10 mins)

Improve the plot based on the following requirements



- Improving the axes and legend labels
- Adding a title for the plot
- Tweaking the color scale to your favorite palette
- The background should be white, not pale grey
- Major gridlines should be a pale grey and minor gridlines should be removed
- The plot title should be centered in 12pt bold text

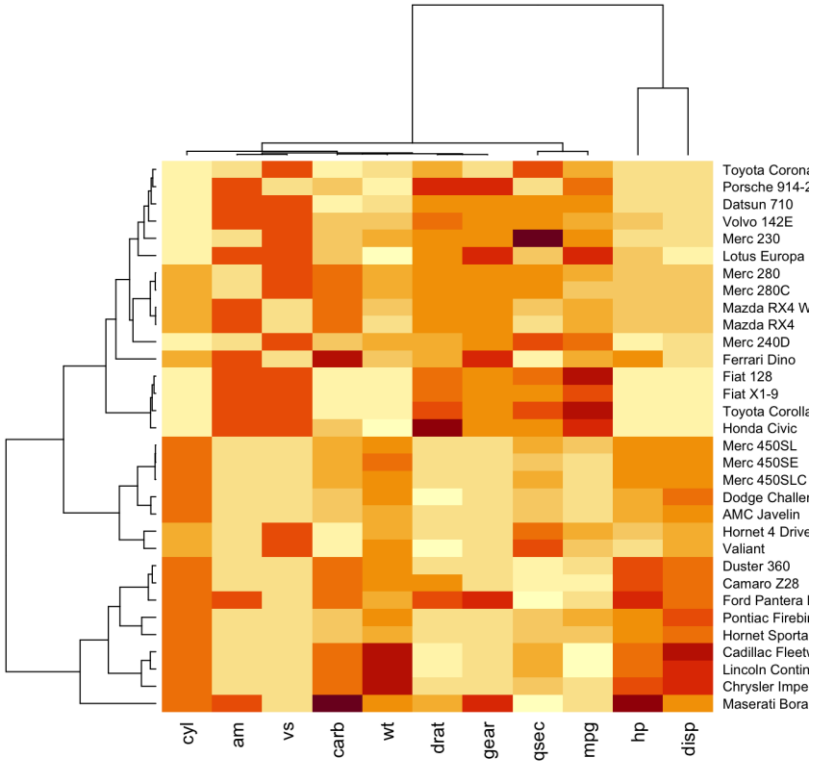


Specialized Plots

Heatmap

1. Base R: heatmap() Function
2. ggplot2 with geom_tile()
3. pheatmap for Annotated Heatmaps

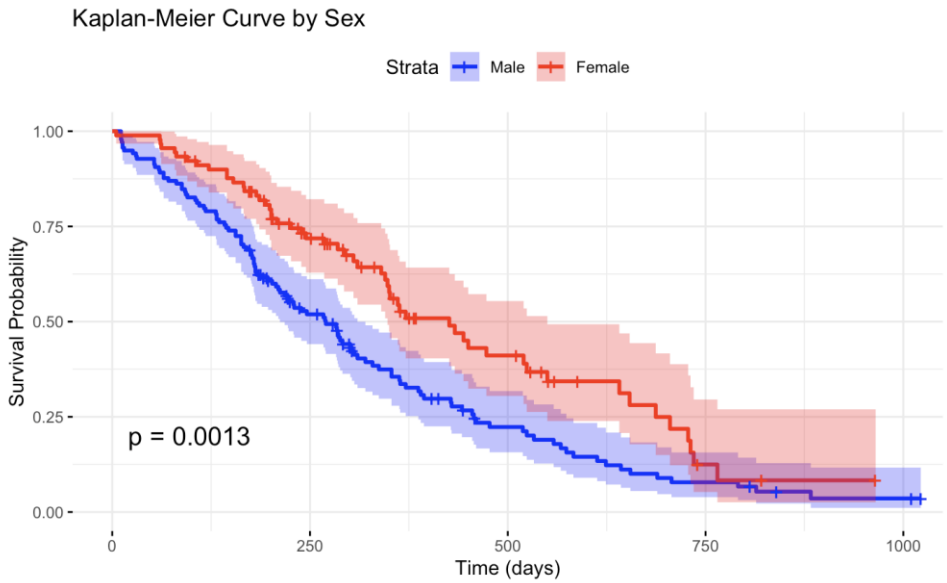
- Correlation analysis
- Feature importance in Machine Learning
- Geospatial data
- Gene expression data in biology
- Resource utilization
- Data quality checks
- Comparing performance metrics
- Cluster analysis



Useful materials: https://r-graph-gallery.com/215-the-heatmap-function.html

Kaplan-Meier curve

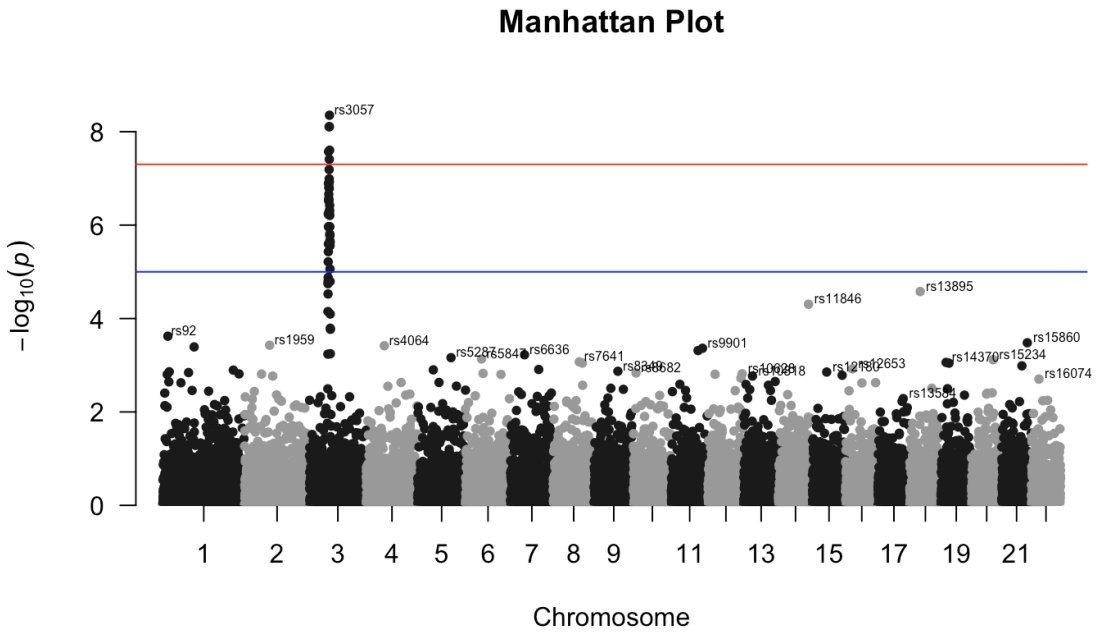
- Survival analysis
- Estimate survival probability over time



"ggsurvplot()" from the survminer library is a useful function to generate KM curves using ggplot2

Manhattan plot

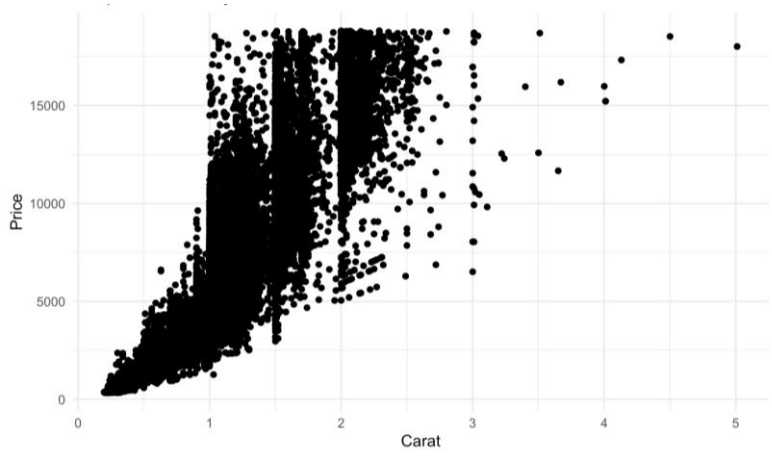
- Used primarily in genome-wide association studies (GWAS) to visualize p-values across genomic positions.



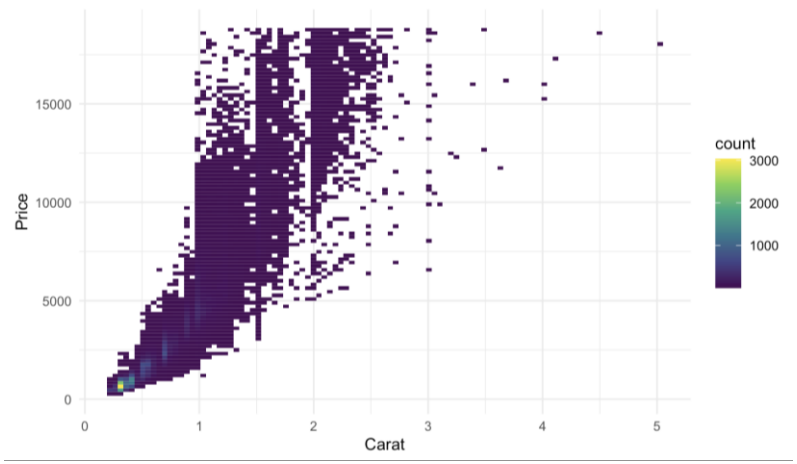
Useful materials: https://r-graph-gallery.com/101_Manhattan_plot.html

2D density plot

- A 2D density plot is a useful visualization to understand the distribution of data points over two continuous variables if you have a huge number of points.



geom_bin2d()



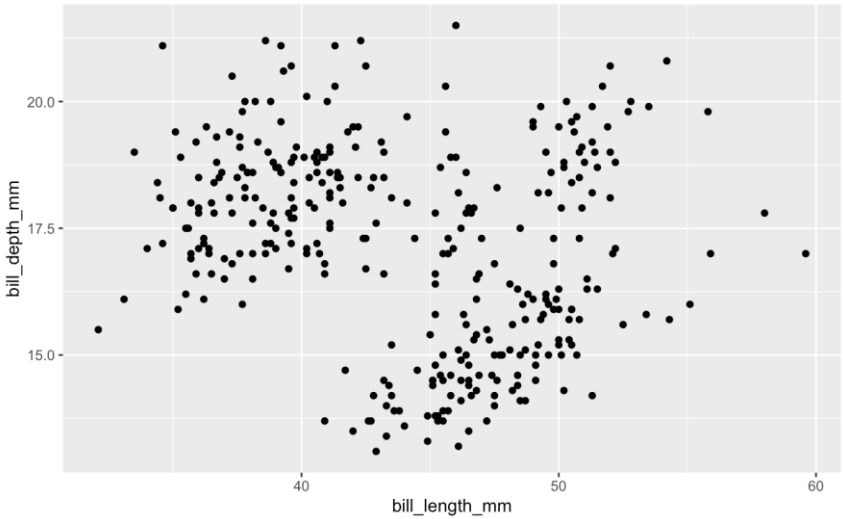
Useful materials: https://r-graph-gallery.com/2d-density-plot-with-ggplot2.html



Interactive Plots



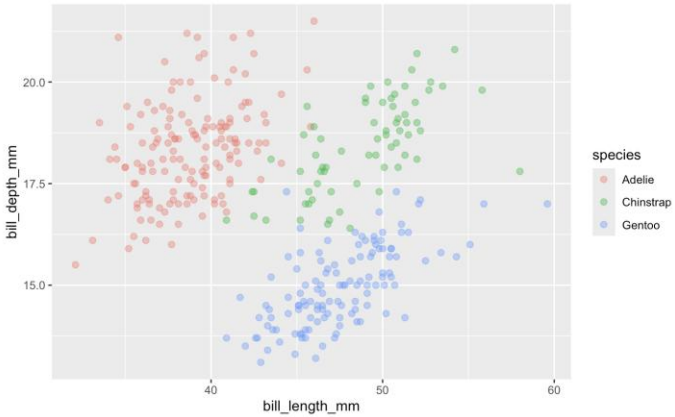
- What can you observe from this plot?
- Do you have any questions about this plot?



Check the data

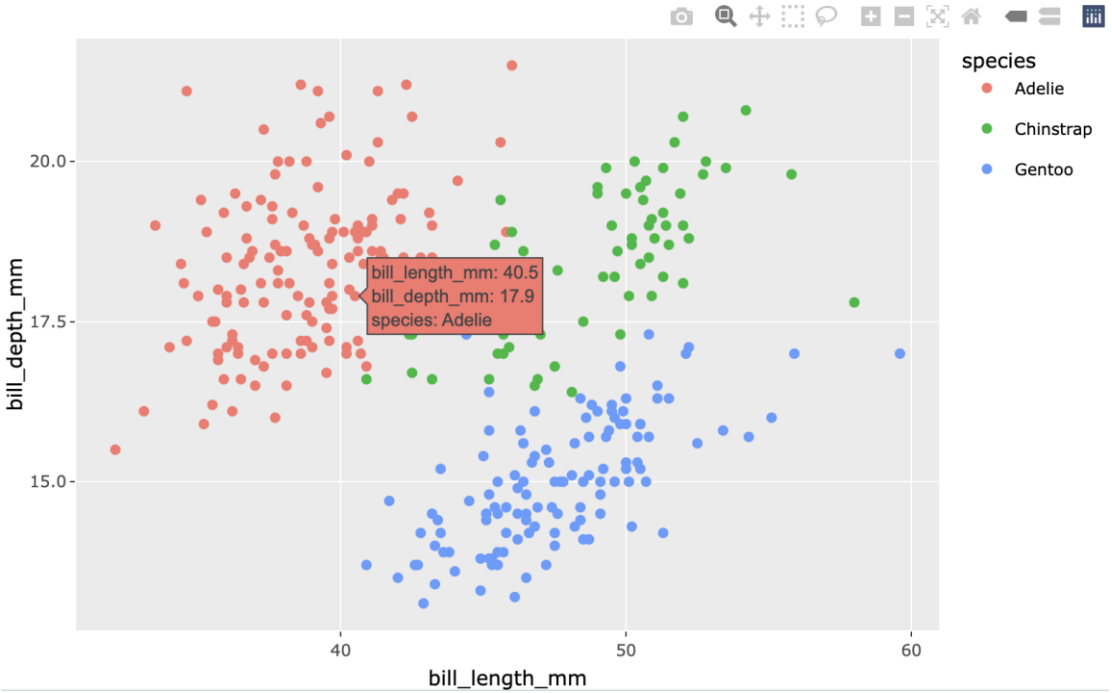
- Any variables that could be used to further explain this plot?
- Species/ Island?

species <fctr>	island <fctr>	bill_length_mm <dbl>	bill_depth_mm <dbl>	flipper_length_mm <int>	body_mass_g <int>	sex <fctr>	year <int>
Adelie	Torgersen	39.1	18.7	181	3750	male	2007
Adelie	Torgersen	39.5	17.4	186	3800	female	2007
Adelie	Torgersen	40.3	18.0	195	3250	female	2007
Adelie	Torgersen	36.7	19.3	193	3450	female	2007
Adelie	Torgersen	39.3	20.6	190	3650	male	2007
Adelie	Torgersen	38.9	17.8	181	3625	female	2007



What if we are still interested in individual info?

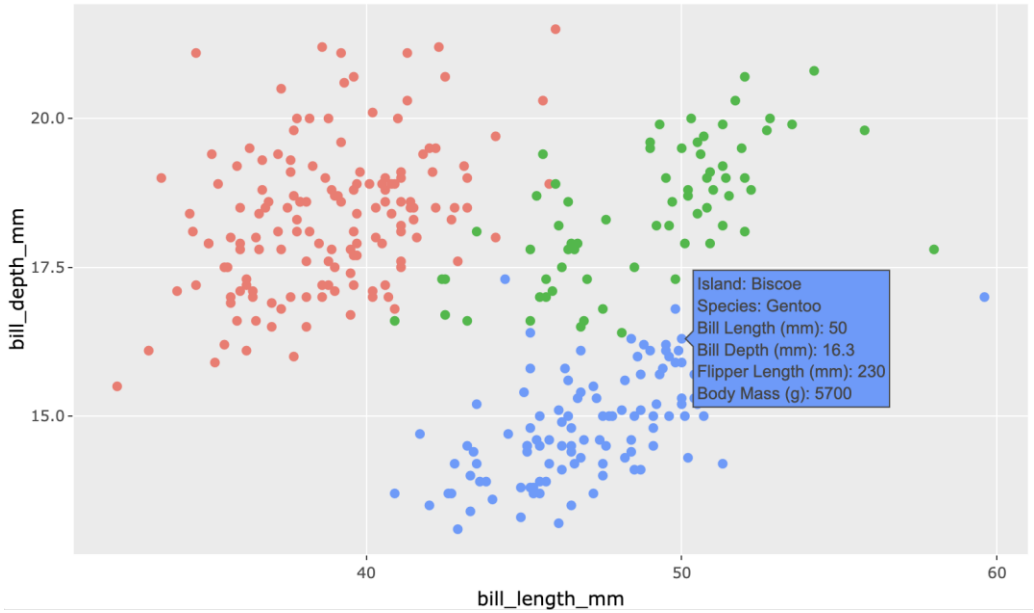
Try ggplotly!



What if we are interested in which island the penguin lives or any other information?

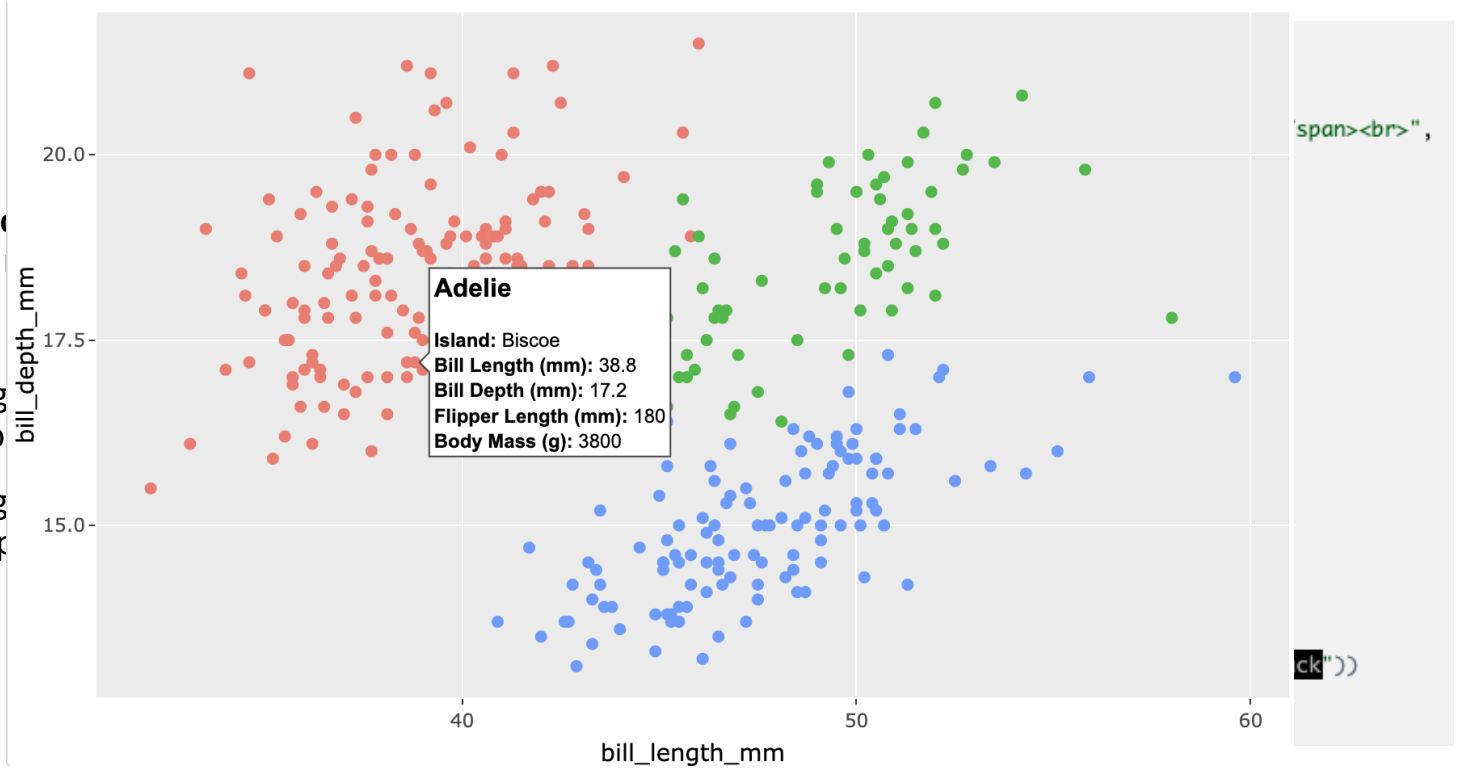
Customize tooltip

```
# customize tooltip
penguins = penguins %>%
  mutate(text = paste(
    "Island: ", island,
    "\nSpecies: ", species,
    "\nBill Length (mm): ", bill_length_mm,
    "\nBill Depth (mm): ", bill_depth_mm,
    "\nFlipper Length (mm): ",flipper_length_mm,
    "\nBody Mass (g): ", body_mass_g, sep=""
  ))
p = ggplot(penguins) +
  geom_point(aes(x = bill_length_mm,
                 y = bill_depth_mm,
                 color = species,
                 text = text)) +
  theme(legend.position = "none")
ggplotly(p, tooltip="text")
```



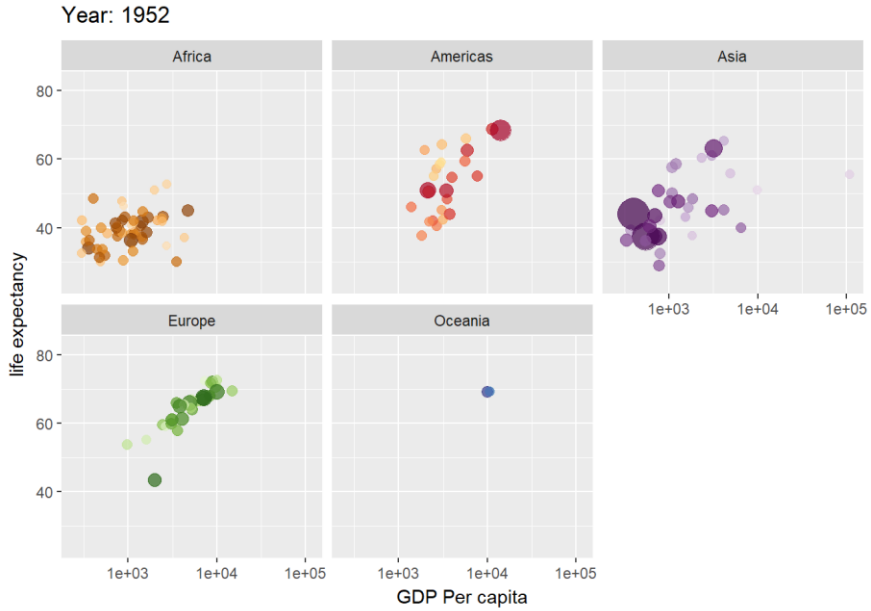
More advanced customization

- Use **b**
  more
- Add a
- Chang
  the to
- Chang
  tooltip



Animate your figure

Try gganimate!



https://gganimate.com/

R Shiny Dashboard

- Shiny is an R package used to build **interactive web applications** directly from R
- Common use cases: data visualization, dashboards, interactive reports, and real-time data processing.
- **Structure**: UI (User Interface) + server (backend logic)
- **UI design**: fluidPage(), sidebarLayout(), ……
- Shiny is reactive, which means that inputs trigger changes in the outputs

Let’s first see an example of R shiny dashboard to analyze Omics Data - GraphBio

http://www.graphbio1.com/en/

Skeleton of R Shiny

User interface (UI)

[define visual elements]

+

Server

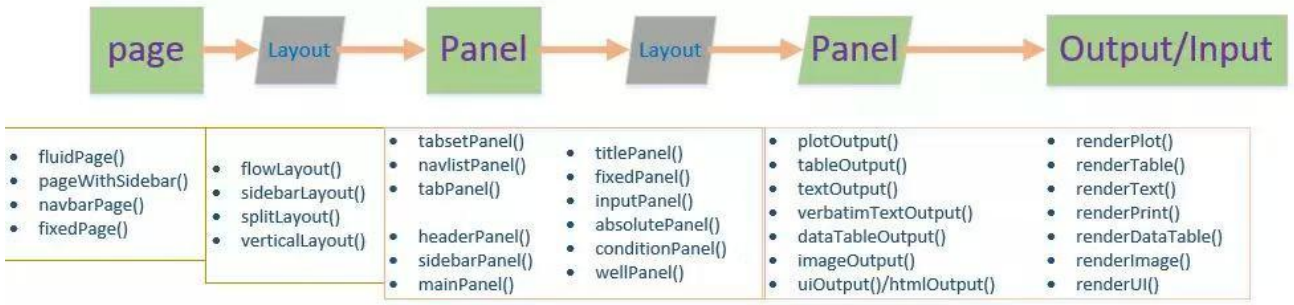
[handle the reactive behavior and interactions]

```
library(shiny)


ui <- fluidPage(
  # Define UI components here
  )

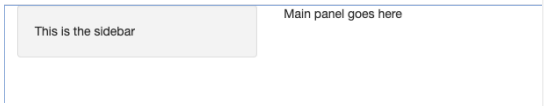
server <- function(input, output){
  # Implement server logic here
}

shinyApp(ui = ui, server = server)
```

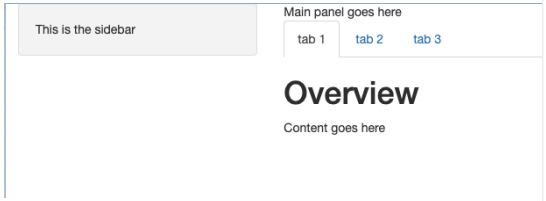


Layout options: Layout/Panel

- SidebarLayout



- TabsetPanel




```
ui <- fluidPage(
  # Define UI components

  sidebarLayout(

    sidebarPanel(
      "This is the sidebar"
    ),

    mainPanel(
      "Main panel goes here",

      tabsetPanel(
        tabPanel(title = "tab 1",
                 h1("Overview"),
                 "Content goes here"),
        tabPanel(title = "tab 2", "The content of the second tab"),
        tabPanel(title = "tab 3", "The content of the third tab")
      )
    )
  )
```

Placeholders

- Define a placeholder for plots, tables, and text in the user interface and server side.

- Plot
- Table
- Text

```
# ui
plotOutput(outputId = "plot"),
tableOutput(outputId = "table"),
textOutput(outputId = "text")

# ----------------------------

# server
output$plot <- renderPlot({
                  plot(x, y, ...)
})

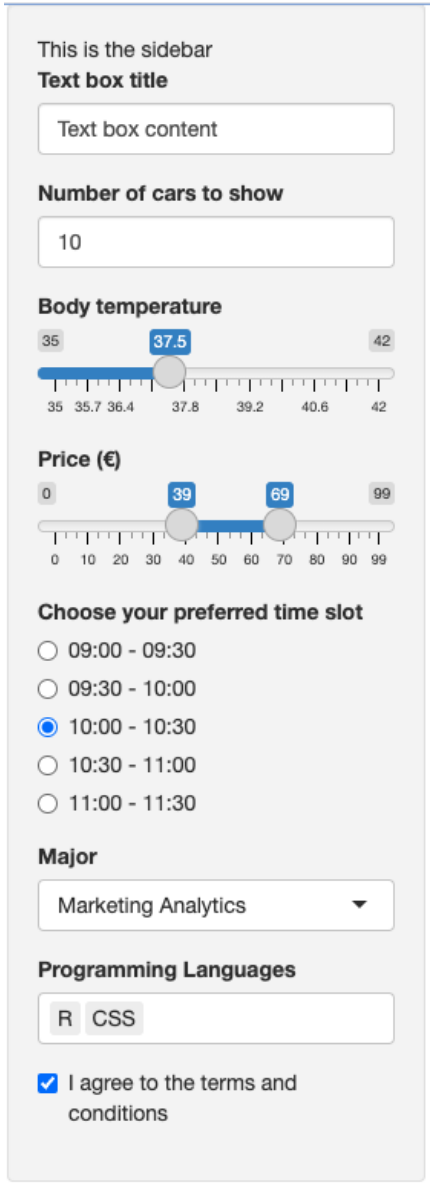
output$table <- renderTable({
                  data
})

output$text <- renderText({
              "Some text"
})
```

Control Widgets

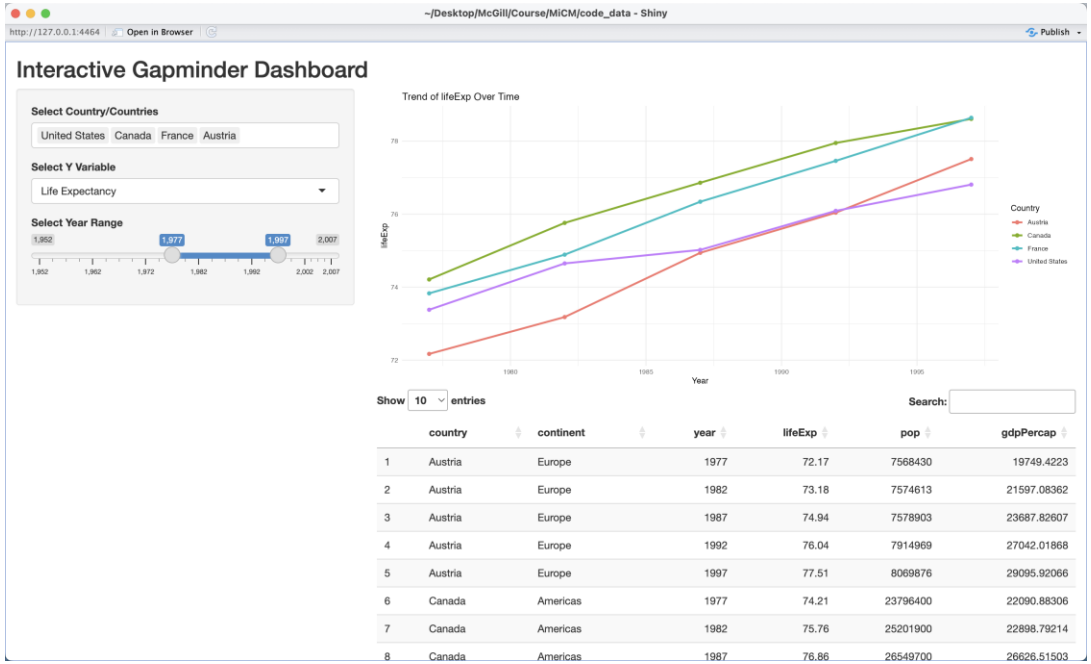
- Allow users to **interact** with the app

- Text box
- Numeric input
- Slider
- Range selector
- Radio button
- Dropdown menu (single selection/multiple selections)
- Checkbox
- ……

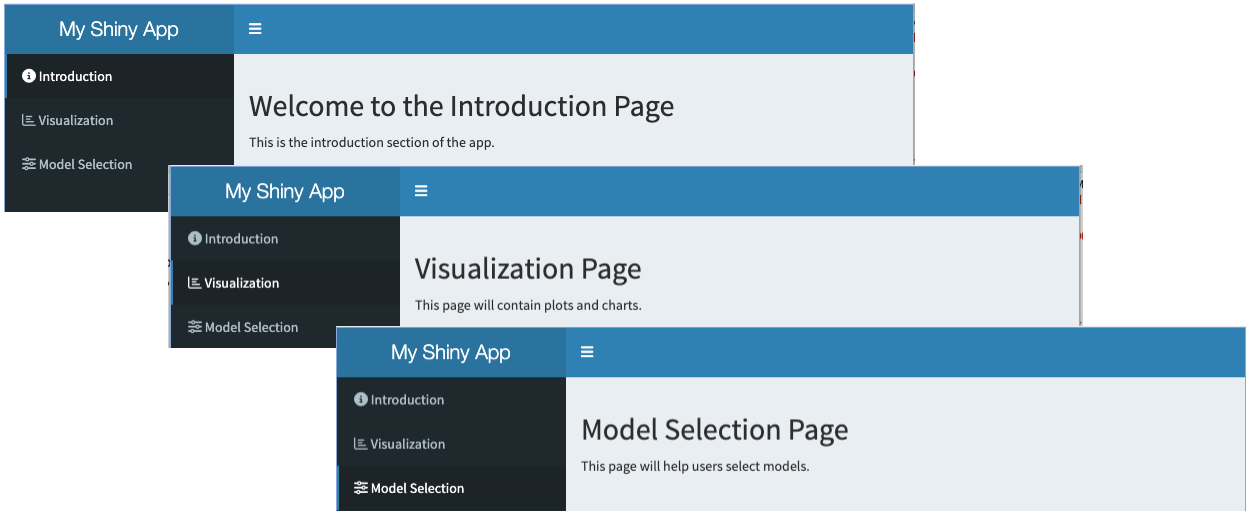


More widgets: https://shiny.posit.co/r/gallery/

Let’s work on a simple R shiny demo!



How to arrange shiny in multiple pages



Use dashboardPage() from shinydashboard package in UI! Another choice: navbarPage()

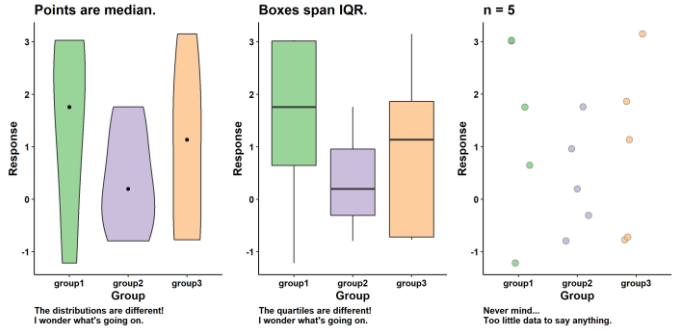
Check "3_ShinyDemo.R" to see how to combine "1_ShinyDemo.R" and "2_ShinyDemo.R" into one Shiny app.

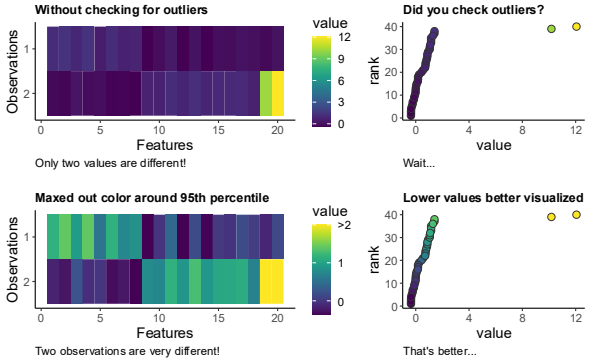
Exercise 2 (15 mins)

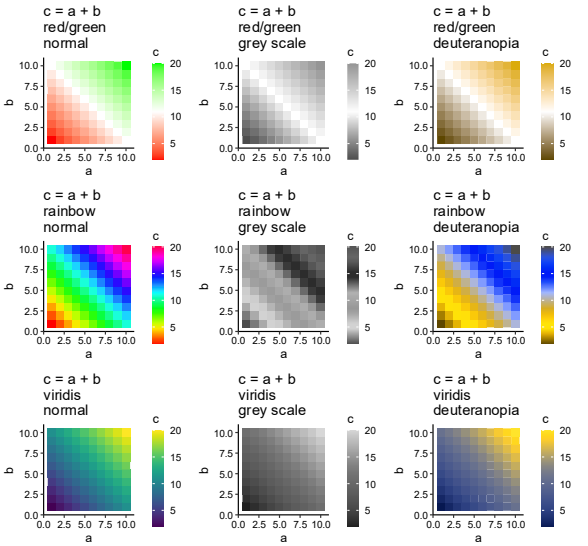
Please select one of the example plots and use ggplotly to make it interactive. If you have time, try to create your own R shiny dashboard!

Friends don’t let friends make bad graphs

- Don’t make violin plots for small sample sizes
- Don’t make heatmap without checking outliers
- Don’t use red/green and rainbow color scales
- ……







https://github.com/cxli233/FriendsDontLetFriends

McGill initiative in Computational Medicine

Useful resources

- ggplot2:
  - Elegant Graphics for Data Analysis (3e) https://ggplot2-book.org/
  - https://ggplot2.tidyverse.org/reference/
- More about data visualization in R (including code, very useful!)
  - https://r-graph-gallery.com/
- Fundamentals of Data Visualization:
  - https://clauswilke.com/dataviz/
- Friends don't let friends make bad graphs:
  - https://github.com/cxli233/FriendsDontLetFriends
- R shiny:
  - https://shiny.posit.co/r/gallery/
  - https://shiny.posit.co/r/getstarted/shiny-basics/lesson1/
  - https://bookdown.org/weicheng/shinyTutorial/
  - https://www.appsilon.com/post/r-shiny-in-life-sciences-examples

Thank you for attending!

Let us know what you think!

